Cuaderno de problemas
Fundamentos de algorítmia.

Algoritmos iterativos

Prof. Isabel Pita

17 de octubre de 2024

Índice

1. Número de máximos. **3**
- 1.1. Objetivos del problema . . . 4
- 1.2. Ideas generales. . . . 4
- 1.3. Algunas cuestiones sobre implementación. . . . 4
- 1.4. Ideas detalladas. . . . 4
- 1.5. Coste de la solución . . . 4
- 1.6. Errores frecuentes. . . . 5
- 1.7. Implementación en C++. . . . 5

2. ¿Esta bien dividido el vector? **7**
- 2.1. Objetivos del problema . . . 8
- 2.2. Ideas generales. . . . 8
- 2.3. Algunas cuestiones sobre implementación. . . . 8
- 2.4. Coste de la solución . . . 8
- 2.5. Errores frecuentes. . . . 9
- 2.6. Implementación en C++. . . . 9

3. Recorrido con acumuladores. Evolución de beneficios. **11**
- 3.1. Objetivos del problema . . . 13
- 3.2. Ideas generales. . . . 13
- 3.3. Algunas cuestiones sobre implementación. . . . 13
- 3.4. Ideas detalladas. . . . 13
- 3.5. Especificación . . . 13
- 3.6. Coste de la solución . . . 14
- 3.7. Errores frecuentes. . . . 14
- 3.8. Modificaciones al problema. . . . 14
- 3.9. Implementación en C++. . . . 14

4. Vector de acumulados. Reserva de elefantes **16**
- 4.1. Objetivos del problema . . . 18
- 4.2. Ideas generales. . . . 18
- 4.3. Algunas cuestiones sobre implementación. . . . 18
- 4.4. Ideas detalladas. . . . 18
- 4.5. Especificación . . . 19
- 4.6. Coste de la solución . . . 20
- 4.7. Errores frecuentes. . . . 20
- 4.8. Implementación en C++. . . . 20

5. Desplazar elementos de un vector. Sensores defectuosos. **22**
- 5.1. Objetivos del problema . . . 23
- 5.2. Ideas generales. . . . 23
- 5.3. Algunas cuestiones sobre implementación. . . . 23
- 5.4. Ideas detalladas. . . . 23
- 5.5. Especificación e invariante del bucle implementado . . . 24
- 5.6. Coste de la solución . . . 25
- 5.7. Errores frecuentes. . . . 25
- 5.8. Implementación en C++. . . . 25

6. Segmento máximo, rescate aéreo. **27**
- 6.1. Objetivos del problema . . . 28
- 6.2. Ideas generales. . . . 28
- 6.3. Ideas detalladas. . . . 28
- 6.4. Especificación e invariante del bucle implementado. . . . 29
- 6.5. Coste de la solución . . . 30
- 6.6. Errores frecuentes. . . . 31

6.7. Modificaciones al problema. . . . 31
6.8. Implementación en C++. . . . 31

7. Mezcla ordenada. Un móvil nuevo **33**
7.1. Objetivos del problema . . . 34
7.2. Ideas generales. . . . 34
7.3. Algunas cuestiones sobre implementación. . . . 34
7.4. Ideas detalladas. . . . 34
7.5. Especificación e invariante del bucle implementado. . . . 35
7.6. Coste de la solución . . . 37
7.7. Errores frecuentes. . . . 38
7.8. Modificaciones al problema. . . . 38
7.9. Implementación en C++. . . . 38

8. Cálculo de la moda. Juego de dados **40**
8.1. Objetivos del problema . . . 41
8.2. Ideas generales. . . . 41
8.3. Algunas cuestiones sobre implementación. . . . 41
8.4. Ideas detalladas. . . . 41
8.5. Especificación e invariante del bucle implementado. . . . 41
8.6. Coste de la solución . . . 43
8.7. Errores frecuentes. . . . 43
8.8. Implementación en C++. . . . 43

9. Cálculo de la moda. Mejorando las carreteras **45**
9.1. Objetivos del problema . . . 46
9.2. Ideas generales. . . . 46
9.3. Algunas cuestiones sobre implementación. . . . 46
9.4. Ideas detalladas. . . . 46
9.5. Coste de la solución . . . 46
9.6. Errores frecuentes. . . . 47
9.7. Implementación en C++. . . . 47

10. Intervalos, releyendo un libro. **49**
10.1. Objetivos del problema . . . 50
10.2. Ideas generales. . . . 50
10.3. Algunas cuestiones sobre implementación. . . . 50
10.4. Ideas detalladas. . . . 50
10.5. Coste de la solución . . . 51
10.6. Errores frecuentes. . . . 51
10.7. Modificaciones al problema. . . . 51
10.8. Implementación en C++. . . . 52

11. Partición. Viajes a Marte. **53**
11.1. Objetivos del problema . . . 55
11.2. Ideas generales. . . . 55
11.3. Algunas cuestiones sobre implementación. . . . 55
11.4. Ideas detalladas. . . . 55
11.5. Coste de la solución . . . 56
11.6. Errores frecuentes. . . . 56
11.7. Implementación en C++. . . . 56

12.Partición. Cintas de colores. **58**
12.1. Objetivos del problema 60
12.2. Ideas generales. 60
12.3. Algunas cuestiones sobre implementación. 60
12.4. Ideas detalladas. 61
12.5. Coste de la solución 61
12.6. Errores frecuentes. 61
12.7. Implementación en C++. 61

1. Número de máximos.

Número de máximos



Hemos desarrollado un juego online y tenemos en un fichero las puntuaciones de diversos jugadores. Ahora queremos saber cuál ha sido la puntuación máxima obtenida y cuántos jugadores la han obtenido. Ha de tenerse en cuenta que el juego permite tener puntuaciones negativas cuando el jugador pierde más puntos de los que obtiene, pero ningún jugador termina con cero puntos.

El problema es que el juego ha tenido tanto éxito que no es posible cargar todo el fichero en memoria, por lo tanto tendremos que resolver el problema al tiempo que vamos leyendo el fichero.

Requisitos de implementación.
No se pueden almacenar los valores de entrada en memoria.

Entrada

La entrada comienza con un valor entero que indica el número de casos de prueba. Cada caso de prueba consta de una línea con las puntuaciones obtenidas por los jugadores. Cada caso termina con el valor 0.

El número de puntuaciones es mayor que cero y no puede suponerse un límite superior. Las puntuaciones son números enteros diferentes de cero y se sabe que pueden almacenarse en una variable de tipo `int`.

Salida

Para cada caso de prueba se escribe en una línea el valor máximo y el número de veces que se repite separados por un carácter blanco.

Entrada de ejemplo

```
3
5 7 -3 4 7 -3 -2 6 5 4 7 1 2 7 3 0
6 6 6 0
5 0
```

Salida de ejemplo

```
7 4
6 3
5 1
```

1.1. Objetivos del problema

- Practicar el uso de variables para guardar información necesaria para resolver el problema.

1.2. Ideas generales.

- Se debe declarar una variable que almacene el valor máximo encontrado entre los valores que se van leyendo y otra que guarde el número de veces que aparece el máximo.
- El máximo o mínimo de una colección de valores debe pertenecer a la colección. Por lo tanto, la variable que guarda el máximo se debe inicializar al primer valor de entrada. Este valor siempre pertenece a la colección.
- Si el enunciado no garantizase que la secuencia de entrada tiene al menos un elemento habría que considerar la entrada sin datos como un caso especial y resolverla según lo que se indique en el enunciado. La solución de inicializar el máximo al elemento mínimo del tipo, si este existe (hay tipos de datos que no tienen un valor máximo o mínimo del tipo, por ejemplo las cadenas de caracteres), puede dar lugar a errores. En cualquier caso se deben utilizar las constantes que definen el elemento mínimo, nunca el valor concreto ya que el mínimo del tipo puede ser dependiente de la máquina que se esté utilizando.

1.3. Algunas cuestiones sobre implementación.

- En el problema se requiere que no se almacenen los datos. Por lo tanto se debe resolver el problema al tiempo que se lee la entrada.
- La función `resuelveCaso` realizará un bucle con centinela para leer los datos de cada caso. Se lee el valor inicial y se ejecuta el bucle mientras el valor leído no sea el centinela. La última instrucción del bucle debe ser la lectura del siguiente dato.

```
// Esquema del bucle de lectura con centinela
    std::cin >> n;
    while (n != centinela) {
        // Codigo que resuelve el caso
        std::cin >> n;
    }
```

1.4. Ideas detalladas.

- Se inicializan las variables que almacenan el valor máximo y el número de veces que aparece. Se lee el valor inicial de la secuencia de entrada
- Se ejecuta un bucle mientras el valor de entrada no sea el centinela.
- En cada vuelta del bucle si el valor es mayor que el máximo que se tiene guardado se modifica el valor del máximo y el número de veces que aparece se inicializará a uno. Si el valor es igual al máximo se incrementa el número de veces que aparece. Si el valor es menor que el máximo no se hace ninguna acción.
- Se lee el siguiente valor de entrada para tratarlo en la siguiente vuelta del bucle.

1.5. Coste de la solución

- En la solución del problema se recorrerá la secuencia de entrada una única vez y en cada iteración del bucle realizaremos en el caso peor uno o dos condicionales y una o dos asignaciones. Realizamos 4 instrucciones en cada iteración todas ellas de coste constante, por lo que el coste está en el orden de $\mathcal{O}(n)$ siendo n el número de puntuaciones de la entrada de datos.

1.6. Errores frecuentes.

- Los valores de entrada pueden ser todos negativos, si se inicializa el valor del máximo a cero, el valor del máximo no se modifica en el bucle, dando como resultado un máximo que no pertenece a la secuencia de datos de entrada. El máximo se debe inicializar al primer valor de la secuencia de entrada para que su valor pertenezca a la colección de valores.
- Utilizar la función `sort` para resolver el problema. Esta solución no es eficiente. Para encontrar el máximo de una colección de elementos no es necesario ordenarlos. La operación de ordenar tiene complejidad $\mathcal{O}(n \log n)$ siendo n el número de elementos por lo que ordenar la colección de valores no es eficiente ya que el problema se puede resolver con complejidad lineal en el número de elementos.
- No usar el esquema de lectura de datos con centinela y realizar la lectura del dato al empezar el bucle. En este caso se ejecutan las instrucciones del bucle con el valor centinela y si todos los datos son negativos resultará como máximo el centinela que no debe procesarse. Para evitarlo se debe realizar una lectura antes del bucle, que nos da la inicialización del valor máximo, y otra lectura como instrucción final del bucle, de forma que después de leer se compruebe la condición del bucle y este finalice si se trata del valor centinela.

1.7. Implementación en C++.

```
#include <iostream>
#include <fstream>


void resuelveCaso() {
    // Inicializacion de variables
   int n;
   std::cin >> n;
    int maxi = n; int numVeces = 1;
    // Bucle que resuelve el problema
    std::cin >> n;
    while (n != 0) {
        if (maxi < n) {
            maxi = n; numVeces = 1;
        }
        else if (maxi == n) ++numVeces;
        std::cin >> n;
    }
    // Salida de datos
    std::cout << maxi << ' ' << numVeces << '\n';
    return true;
}

int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif
   // Entrada con numero de casos
    int numCasos;
    std::cin >> numCasos;
    for (int i = 0; i < numCasos; ++i) resuelveCaso()
        ;

    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
   #endif
```

```
    return 0;
}
```

2. ¿Esta bien dividido el vector?

¿Está bien dividido el vector?

Nos dan un vector de números enteros con n elementos y una cierta posición p del vector. Debemos comprobar si todas las componentes del vector en las posiciones [0..p] (incluida la p) son menores (estrictamente) que todas las componentes en las posiciones [p+1..n-1].

Si *p=n-1* el resultado debe ser cierto.

Requisitos de implementación.

Se implementará una función que reciba un vector con los valores y el valor p y que devuelva un valor booleano que indique si el vector está bien dividido.

El orden de complejidad del algoritmo debe ser lineal respecto al número de elementos del vector.

Entrada

La entrada comienza con un valor entero que indica el número de casos de prueba. Cada caso de prueba consta de dos líneas. La primera indica el número de elementos del vector y la posición que debería separar los valores menores de los mayores. La segunda línea contiene los valores del vector.

El número de valores de entrada n cumple: $1 \leq$ n ≤ 300.000 y la posición p cumple $0 \leq p \leq n$-1. Los valores del vector son números enteros que pueden almacenarse en una variable de tipo `int`.

Salida

Para cada caso de prueba se escribe en una línea *SI* si la posición dada separa el vector en la forma apropiada y *NO* si no lo hace.

Entrada de ejemplo

```
5
15 5
5 7 3 4 7 3 8 9 8 10 8 9 8 9 10
4 0
4 7 5 6
4 0
8 7 5 6
4 3
4 3 6 2
5 2
5 3 4 6 4
```

Salida de ejemplo

```
SI
SI
NO
SI
NO
```

Autor: Isabel Pita

2.1. Objetivos del problema

- Practicar el uso de variables para guardar información que ayude a resolver el problema.
- Practicar el tipo `vector`.

2.2. Ideas generales.

- En el enunciado del problema nos indican que nos dan un vector. Por lo tanto el problema debe resolverse en una función que reciba como parámetro de entrada un vector con los datos del problema y devuelva el resultado. La función `resuelveCaso`, se ocupa de leer los datos de entrada, llamar a la función que resuelve el problema y escribir los resultados.
- Observad que todos los valores de una colección C1 son menores que los valores de otra colección C2 si el máximo de la colección C1 es menor que el mínimo de la colección C2.
- El problema se puede resolver de varias formas:
  - Calcular el máximo de los elementos del intervalo [0..p], a continuación comprobar que este elemento es menor que todos los elementos del intervalo (p..n).
  - Calcular el máximo de los elementos del intervalo [0..p] y el mínimo de los valores del intervalo (p..n). A continuación comparar ambos valores.
  - Realizar un tratamiento semejante a cualquiera de los anteriores pero recorriendo el vector de derecha a izquierda.
  - En los tres casos el coste asintótico de los algoritmos es lineal en el número de datos de la entrada. En el primer caso el algoritmo puede terminar sin recorrer todos los elementos del vector, mientras que en el segundo caso siempre se recorre el vector completo sin que influya el valor concreto de los valores guardados en el vector.

2.3. Algunas cuestiones sobre implementación.

- *Paso de parámetros.*

  La función recibe el vector de datos y no lo modifica, por lo tanto el parámetro es `const&`. La función recibe además el valor de `p`, este parámetro puede ser por *valor* ya que su tipo es `int`.
- El problema debe resolverse utilizando dos bucles en secuencia. El primero recorre el vector hasta la posición `p` y el segundo desde la posición `p+1`. No tiene ninguna ventaja utilizar un único bucle que pregunte en cada ejecución si ya se ha alcanzado el valor de `p`, se realizan más operaciones y el código es más complicado.

2.4. Coste de la solución

- En la solución del problema se implementan dos bucles: uno para calcular el máximo hasta la posición `p` y otro para comprobar que el valor no es mayor que los elementos restantes. Las instrucciones de ambos bucles tienen coste constante. Como el número de veces que se ejecutan las instrucciones de ambos bucles es el número de elementos del vector, y este es el número de datos de entrada, el coste en el caso peor está en $\mathcal{O}(n)$ siendo n el número de datos de la entrada.
- Si el problema se implementa calculando el máximo de la parte izquierda y el mínimo de la parte derecha el coste asintótico en el caso peor es el mismo que calculando solo el máximo y comparando este valor con el resto del vector. Sin embargo, aunque el coste asintótico sea el mismo, en la implementación en que solo se calcula el máximo, el bucle finaliza cuando se encuentra un valor menor que el máximo en la parte derecha. Esto permite en algunos casos no recorrer todo el vector.
- Sin embargo, en el caso peor, que se produce cuando el vector está bien dividido, ambas implementaciones tienen el mismo coste asintótico y ambas recorren todos los elementos del vector.

2.5. Errores frecuentes.

- Realizar una copia del vector en el paso de parámetros a la función.
- No calcular correctamente el máximo o el mínimo del intervalo correspondiente por inicializar estas variables a valores que no pertenecen al vector. En concreto no se debe inicializar el máximo a cero, ya que el vector puede ser todo de valores negativos.
- Realizar una implementación de coste cuadrático respecto al número de elementos del vector. Si cada elemento de la parte izquierda lo comparamos con todos los elementos de la parte derecha, estamos realizando $p*(n-p)$ operaciones. Como `p` es un índice del vector, se puede expresar en función de `n`. Por ejemplo puede ser $\frac{n}{2}$ o $\frac{n}{3}$. El número de operaciones cuando $p = \frac{n}{2}$ es $\frac{n}{2}*(n-\frac{n}{2}) = \frac{n^2}{4} \in \mathcal{O}(n^2)$.

2.6. Implementación en C++.

```
#include <iostream>
#include <fstream>
#include <vector>

bool resolver(std::vector<int> const& v, int p) {
    // recorre buscando el maximo hasta p
    int maximo = v[0];
    int i;
    for (i = 1; i <= p; ++i)
        if (maximo < v[i]) maximo = v[i];
    // recorre comparando con la parte derecha
    while (i < v.size() && v[i] > maximo) ++i;
    return i == v.size();
}

bool resuelveCaso() {
// Leer los datos
    int numElems = 0; size_t p;
    std::cin >> numElems >> p;
    std::vector<int> v(numElems);
    for (int& i : v) std::cin >> i;
    // Resolver el problema
    bool sol = resolver(v,p);
    // Escribir el resultado
    if (sol) std::cout << "SI\n";
    else std::cout << "NO\n";
    return true;

}

int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif
   // Entrada con numero de casos
    int numCasos;
    std::cin >> numCasos;
    for (int i = 0; i < numCasos; ++i) resuelveCaso()
        ;

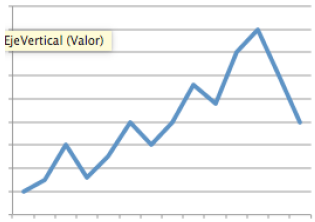
    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
```

```
    #endif

     return 0;
}
```

3. Recorrido con acumuladores. Evolución de beneficios.

Evolución de los beneficios.



Nuestra empresa quiere ampliar el negocio y para ello va a comprar una compañía de fabricación de componentes. Antes de realizar la compra queremos comprobar el estado en que se encuentra la empresa de fabricación y para ello analizamos los datos sobre sus ventas y sus gastos.

Nuestro jefe nos ha encargado que obtengamos los años en que las ventas superaron las ventas de todos los años anteriores. El primer año del que tenemos datos no puede compararse con los anteriores y por lo tanto no lo consideraremos. Para ello contamos con las ventas de cada año desde que se fundó la empresa. Después de dedicar un rato a mirar los números, he decidido que lo más fácil es realizar un programa que me resuelva el problema.

Requisitos de implementación.

Debe implementarse una función que reciba un vector con todos los datos de las ventas y el año en que empezaron a tomarse los datos, y devuelva en otro vector los años en los que se superaron las ventas de los años anteriores.

El coste de la función debe ser del orden del número de datos de entrada.

Entrada

La entrada comienza con un valor entero que indica el número de casos de prueba. Cada caso de prueba consta de dos líneas. En la primera línea se indica el primer y el último año a los que corresponden los valores de las ventas que nos dan. En la segunda línea se muestran los valores de las ventas en los años indicados.

El número de años considerados es mayor que cero y el intervalo está entre los años 1700 y 100.000. Los valores posibles de las ventas son números enteros en el intervalo ($-2^{31}+1$.. $2^{31}-1$).

Salida

Para cada caso de prueba se escriben en una línea los años en los que las ventas superan las ventas de los años anteriores. Si ningún año superó las ventas de los años anteriores se dejará una línea en blanco.

Entrada de ejemplo

```
5
2000 2017
159 172 181 190 201 213 227 239 243 233 232 229 225 225 227 233 241 250
2005 2012
281 292 304 310 300 308 315 318
2013 2017
-120 -140 -60 -50 -70
1990 1992
20 10 5
3500 3503
25 25 27 40
```

Salida de ejemplo

```
2001 2002 2003 2004 2005 2006 2007 2008 2017
2006 2007 2008 2011 2012
2015 2016

3502 3503
```

Autor: Isabel Pita

3.1. Objetivos del problema

- Practicar a utilizar variables acumuladoras para evitar realizar cálculos acumulativos en cada iteración de un bucle.
- Practicar el uso y recorrido de vectores.

3.2. Ideas generales.

- Los valores que se necesita conocer para decidir si el valor de la posición i-ésima es mayor que todos los valores de su izquierda son los valores de la izquierda del vector. Por lo tanto se debe recorrer el vector de izquierda a derecha.
- Para evitar comprobar en cada posición si el valor es mayor que todos los de la izquierda, se guarda en una variable auxiliar el valor máximo encontrado hasta el punto en que se está recorriendo el vector.
- El problema se podría resolver sin necesidad de utilizar un vector para guardar los datos de entrada. En el enunciado se pide usar el vector para aislar el algoritmo de solución del problema del manejo de la entrada/salida de datos y calcular el coste del algoritmo independientemente del coste de lectura de los datos de entrada y para que se practique el uso de la clase vector.

3.3. Algunas cuestiones sobre implementación.

- *Paso de parámetros.*

  La función recibe el vector de datos y no lo modifica, por lo tanto el parámetro es `const&`. La función recibe además el valor de inicio del periodo, este parámetro puede ser por *valor* ya que su tipo es `int`.

- *Valor de retorno.*

  La función devuelve un vector con los años que cumplen la propiedad pedida. Este vector se declara en la función y no se conoce su número de elementos. Por lo tanto se declara vacío y se añaden sus elementos con la función `push_back` definida en la clase `vector`.

3.4. Ideas detalladas.

- *Cómo calcular el número de elementos del vector.*

  Conocemos el año correspondiente al primer valor y el año correspondiente al último valor. Como solo se consideran años positivos, el número de datos es el año final menos el año inicial más uno.

- *Inicializar el valor máximo.*

  Como los datos pueden ser todos negativos, el beneficio máximo debe inicializarse al primer elemento del vector. En el enunciado se garantiza que el vector no será nulo.

3.5. Especificación

La especificación consta de tres partes:

1. La precondición indica las propiedades que cumplen los datos de entrada. En nuestro caso, la función que resuelve el problema tiene dos parámetros de entrada: el vector con los datos de los beneficios de cada año y el valor del año inicial. En el enunciado se indica que el número de años considerados es mayor que cero, por lo que el vector tiene al menos un elemento. La segunda condición indica que el valor del año inicial está entre los años 1700 y 4000 ambos incluidos. Se observa que el año inicial puede coincidir con el año final. La última condición que se indica es que los valores del vector son números enteros que pueden almacenarse en una variable de tipo `int`.

   El valor del año final se utiliza para determinar el tamaño de los datos de entrada, no siendo necesaria para resolver el problema una vez obtenidos los datos. Por esta razón no se considera como precondición de la función que resuelve el problema.

2. La cabecera de la función. En ella indicamos el nombre de la función, los parámetros de entrada o entrada/salida de la función y el tipo del valor devuelto por la función. En nuestro caso tenemos dos parámetros de entrada: el vector y el año inicial y un valor de salida de tipo vector. Observad que se proporciona un identificador tanto a los parámetros de entrada como al valor de salida para poder referirnos a ellos en la precondición y en la postcondición.

3. La postcondición nos muestra las propiedades que cumplen los valores calculados por la función. En nuestro caso, los valores del vector `sol` cumplen que todos los valores a su izquierda son menores que ellos. También debemos comprobar que el vector `sol` tiene todos los valores que cumplen esta propiedad. Para ello comprobamos que si un elemento del vector de entrada `a` no está en el vector `sol` es que no cumple la propiedad.

4. Observad que en la especificación no se refleja el tipo de datos específico de C++ que se debe utilizar para guardar tanto los datos de entrada como los de salida ya que éste es un detalle de implementación. Se indica simplemente que son de tipo entero, utilizaremos *integer* para diferenciarlo del tipo concreto de C++ `int`. En los valores de entrada se debe indicar su rango para que el implementador sepa elegir el tipo adecuado de las variables. En los valores de salida se podría indicar el rango para facilitar la labor del implementador.

$$
\begin{array}{l}
P \equiv \{1 \leq v.size \wedge \forall\, i : 0 \leq i < v.size : -2^{31}+1 \leq v[i] \leq 2^{31}-1 \wedge 1700 \leq ini \leq 4000\} \\
\quad beneficios\ (vector\texttt{<}integer\texttt{>}\ v, integer\ ini)\ dev\ vector\texttt{<}integer\texttt{>}\ sol \\
Q \equiv \{\forall i : 0 \leq i < v.size \wedge i + ini \in sol : (\forall j : 0 \leq j < i : v[j] < v[i] \wedge \\
\qquad \forall i : 0 \leq i < v.size \wedge i + ini \notin sol : (\exists j : 0 \leq j < i : v[j] \geq v[i]\}
\end{array}
$$

3.6. Coste de la solución

En la solución del problema se recorre el vector una única vez y en cada iteración se realiza una pregunta y en el caso peor una asignación y una inserción al final del vector. Realizamos 5 instrucciones en cada iteración todas ellas de coste constante, por lo que el coste está en el orden de $\mathcal{O}(n)$ siendo n el número de elementos del vector.

Añadir un elemento en la última posición de un vector tiene coste amortizado constante. En el caso peor, el vector se llena y hay que hacer una copia de todos sus elementos para poder añadir el nuevo, por lo que en el caso peor el coste es lineal respecto al número de elementos del vector, sin embargo, si al ampliar el vector para añadir un nuevo elemento duplicamos su capacidad, puede demostrarse que si consideramos el coste de realizar varias operaciones este es constante. Aunque en este curso estudiamos el coste de los algoritmos en el caso peor, cuando el coste amortizado de las operaciones sea constante consideraremos el coste como constante.

3.7. Errores frecuentes.

- Realizar una implementación de coste cuadrático respecto al número de elementos del vector. Si para cada elemento calculamos el valor máximo de las componentes anteriores a él, se recorre el vector, hasta la posición que estamos tratando, en cada iteración y por lo tanto el coste es cuadrático en el número de elementos del vector.

3.8. Modificaciones al problema.

- Si la propiedad pedida sobre los datos del vector se refiere a los valores de la derecha del dato, entonces debemos recorrer el vector de derecha a izquierda.

- La propiedad pedida sobre los datos puede referirse al mínimo de los valores anteriores (o posteriores) o a la suma de los valores.

3.9. Implementación en C++.

```cpp
#include <iostream>
#include <fstream>
#include <vector>

std::vector<int> resolver (std::vector<int> const& v, int inicio) {
    std::vector<int> sol;
    int PIBMax = v[0];
    sol.push_back(inicio); // posicion del primer maximo
    for (int k = 1; k < v.size(); ++k) {
        if (v[k] > PIBMax){ // Encontrado nuevo maximo
            PIBMax = v[k];
            sol.push_back(k+inicio);
        }
    }
    return sol;
}

void resuelveCaso() {
    // Lectura de los datos
    int inicio, fin;
    std::cin >> inicio >> fin;
    std::vector<int> v(fin - inicio + 1);
    for (int& n : v) std::cin >> n;
    // Resolver el problema
    std::vector<int> sol = resolver(v, inicio);
    // Escribir los datos de salida
    if (sol.size() > 0) std::cout << sol[0];
    for (int i = 1; i < sol.size(); ++i)
        std::cout << ' ' << sol[i];
    std::cout << '\n';
}

int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif
   // Entrada con numero de casos
    int numCasos;
    std::cin >> numCasos;
    for (int i = 0; i < numCasos; ++i) resuelveCaso()
        ;

    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
   #endif

    return 0;
}
```

4. Vector de acumulados. Reserva de elefantes

Reserva de elefantes



El parque nacional de Chobe, en la región noroeste de Botswana, es famoso por su alta densidad de elefantes. Los encargados del parque llevan muchos años recopilando datos sobre el número de nacimientos de elefantes en la región. Sin embargo, los visitantes del parque no suelen estar interesados en los nacimientos de un determinado año, sino que preguntan por el número de nacimientos entre dos fechas, lo que obliga a los encargados a sumar los datos entre las dos fechas pedidas. Cansados ya de tanto sumar, han decidido que deben mantener las sumas ya realizadas y de esta forma solo tendrán que consultar el dato. Pero mantener todos los intervalos de fechas no resulta viable, por lo que después de pensarlo un poco han decidido que con tener los datos acumulados de los nacimientos de todos los años anteriores es suficiente.

Requisitos de implementación.

Los datos deben prepararse para que resolver cada pregunta tenga un coste constante. La preparación de los datos tiene coste lineal en el número de datos que tiene el parque.

Entrada

La entrada consta de una serie de casos de prueba. Cada caso se escribe en varias líneas. En la primera se muestra el primer y último año (0 < p <= u <= 100 000) en los que se han tomado datos de los nacimientos de elefantes. En la segunda línea se muestran los `n = u-p+1` nacimientos ocurridos durante estos años (0 <= k < 2^{30}). En la línea siguiente se indican el número de preguntas que realizan los visitantes (0 < m <=100 000) y en las `m` líneas siguientes se muestran dos años (p <= a <= b <= u) que representan el intervalo en el que se debe calcular el número de nacimientos de elefantes.

El final de los casos se marca con dos ceros que no deben tratarse.

Salida

Para cada caso de entrada se escriben tantas líneas como preguntas haya en el caso. En cada línea se escribe el número de nacimientos ocurridos durante los años que se indican en la pregunta. Ambos años incluidos.

Cada caso termina con una línea con tres guiones (`---`).

Entrada de ejemplo

```
1950 1955
4 1 2 0 3 6
3
1950 1953
1952 1955
1951 1954
2010 2016
4 2 1 6 4 4 2
2
2014 2014
2010 2016
0 0
```

Salida de ejemplo

```
7
11
6
---
4
23
---
```

Autor: Isabel Pita

4.1. Objetivos del problema

- Crear un vector a partir de otro con sus valores acumulados.
- Obtener información a partir de un vector de valores acumulados. Estas ideas se aplican en la solución de problemas con la técnica de vuelta atrás.
- Recordar los modificadores de tipo de C++.

4.2. Ideas generales.

- En la descripción de la entrada se indica que para los datos obtenidos entre dos fechas se realizarán varias preguntas, y aclara que el número de preguntas puede llegar a 100.000. Este requisito insinúa que la respuesta a cada pregunta se debe realizar en tiempo *casi constante*, ya que el coste de resolver cada caso multiplica el coste de responder cada pregunta por el número de preguntas realizadas. Por lo tanto, se deben preparar los datos de entrada de forma que nos permitan responder a las preguntas *rápido*.
- Nos basaremos en: $\sum_{i=x}^{y} a_i = \sum_{i=0}^{y} a_i - \sum_{a=0}^{x-1} a_i$.
- Se creará un vector, que llamaremos *vector de acumulados*, cuya componente i-ésima tenga el valor $\sum_{j=0}^{i} a_j$.
- A partir de los datos del vector de acumulados se puede obtener mediante una simple operación aritmética el resultado de cada pregunta.

4.3. Algunas cuestiones sobre implementación.

- El número de valores de entrada se obtiene como la diferencia de los años de fin e inicio más uno.
- El vector de acumulados puede crearse con una componente más que el vector de entrada y fijar el valor de la primera componente a cero. De esta forma al restar el año de inicio de toma de datos del inicio del periodo obtenemos el índice del vector correspondiente a la suma de los valores anteriores al inicio del periodo.

  indice suma valores anteriores al inicio del periodo = año inicio periodo - año inicio datos.

  La suma acumulada de los nacimientos desde el inicio de toma de datos hasta el final del periodo incluido lo obtenemos en la componente siguiente a la resta del inicio de la toma de datos del año de finalización.

  suma desde el inicio de datos hasta final del periodo = v[año fin periodo - año inicio datos].

- Si el vector de acumulados se declara con el mismo número de componentes que el vector de entrada, la suma de los valores hasta el comienzo del periodo (sin incluir) está en la posición

  indice suma valores anteriores al inicio del periodo = año inicio periodo - año inicio datos - 1.

  Por lo tanto hay que diferenciar el caso en que el inicio del periodo coincide con el inicio de los datos para no acceder al vector fuera de rango.

4.4. Ideas detalladas.

- El problema hará uso de una función que calcule el vector de acumulados. Esta función recorre el vector de entrada de izquierda a derecha y cada componente se calcula como la componente anterior acumulada mas el valor de la entrada.

```
acum[0] = 0;
for (int i = 1; i < entrada.size() + 1; ++i)
{
    acum[i] = acum[i-1] + entrada[i-1];
}
```

- El último valor del vector de acumulados coincide con la suma de todos los valores del vector. Se debe elegir el tipo de datos adecuado para almacenar la suma teniendo en cuenta el valor máximo que se puede sumar y el número de veces que debemos sumarlo de forma que no haya desbordamiento de la variable.

- Después de calcular el vector de acumulados se procesarán las preguntas. Para cada pregunta se calcula la respuesta haciendo uso del vector y se muestra en consola.

```
int m; std::cin >> m;
for (int i = 0; i < m; ++i) {
  int f1, f2;
  std::cin >> f1 >> f2;
  std::cout << acum[f2-a1+1]-acum[f1-a1] << '\n';
}
```

4.5. Especificación

Planteamos dos especificaciones. La primera especifica el problema pedido: dados los valores de los nacimientos de elefantes durante una serie de años contestar a la pregunta de cuántos han nacido entre dos años determinados. La segunda especifica el algoritmo que dado un vector con valores numéricos calcula el vector de acumulados.

Especificación del problema:

1. La precondición indica las propiedades que cumplen los datos de entrada. En nuestro caso el problema tiene como valores de entrada: los datos de los nacimientos de elefantes, y el primer y último año del periodo que se quiere consultar. En el enunciado se indica que el número de datos de nacimientos de elefantes es mayor que cero, ya que el primer año es menor o igual que el segundo. El periodo del que se pide información debe tener también al menos un año, por el mismo motivo. Por último, los valores de los nacimientos son números enteros.

2. La cabecera de la función. En ella indicamos el nombre del problema, los datos de entrada o entrada/salida y el tipo del valor calculado. En nuestro caso tenemos tres valores de entrada: la secuencia de nacimientos de elefantes que representamos en un vector y el año inicial y final del periodo pedido. Como salida obtenemos el número de nacimientos ocurridos en el periodo. Observad que se proporciona un identificador tanto a los valores de entrada como al valor de salida para poder referirnos a ellos en la precondición y en la postcondición.

3. La postcondición nos muestra las propiedades que cumplen los valores calculados. En nuestro caso, el valor obtenido es la suma de los valores de entrada comprendidos entre los años dados.

4. Observad que en la especificación no se refleja el tipo de datos específico de C++ que se debe utilizar para guardar tanto los datos de entrada como los de salida ya que este es un detalle de implementación. Se indica simplemente que son de tipo entero, utilizaremos *integer* para diferenciarlo del tipo concreto de C++ `int`. En los valores de entrada se debe indicar su rango para que el implementador sepa elegir el tipo adecuado de las variables. En los valores de salida se podría indicar el rango para facilitar la labor del implementador.

$$P \equiv \{1 \leq v.size \wedge \forall\, i : 0 \leq i < v.size : 0 \leq v[i] < 2^{30} \wedge 0 \leq ini \leq 100000\}$$
$$reservaelefantes\ (vector\texttt{<}integer\texttt{>}\ v, integer\ ini, integer\ fin)\ dev\ integer\ sol$$
$$Q \equiv \{sol = \textstyle\sum_{i=ini}^{i=fin} v[i]\}$$

Especificación del algoritmo que calcula el vector de acumulados:

1. La precondición indica las propiedades que cumplen los parámetros de entrada a la función. En nuestro caso el problema tiene como parámetro de entrada un vector con los datos de los nacimientos de elefantes. Dicho vector tiene al menos un elemento y sus valores son números enteros. Se indica el rango de las variables para que el implementador pueda seleccionar adecuadamente el tipo de las variables que utiliza.

2. La cabecera de la función. En ella indicamos el nombre del problema, los datos de entrada o entrada/salida y el tipo del valor calculado. En nuestro caso tenemos como entrada la secuencia de nacimientos de elefantes que representamos en un vector. Como salida obtenemos el vector con los valores acumulados. Observad que se proporciona un identificador tanto a los valores de entrada como al valor de salida para poder referirnos a ellos en la precondición y en la postcondición.

3. La postcondición nos muestra las propiedades que cumplen los valores calculados. En nuestro caso, se indica que el valor de cada componente del vector solución es la suma de los valores anteriores, incluyendo al propio valor, del vector de entrada.

4. En la especificación se indica que el valor obtenido es el vector con los valores acumulados. Es labor del implementador decidir si el vector resultado de la función se devolverá como parámetro de salida de la función o como valor de retorno. Debe implementar la función de la forma más eficiente para el lenguaje de programación que esté utilizando.

$$
\begin{array}{l}
P \equiv \{1 \leq v.size \wedge \forall\ i : 0 \leq i < v.size : 0 \leq v[i] < 2^{30}\} \\
\ \ acumulados\ (vector\texttt{<}integer\texttt{>}\ v)\ dev\ vector\texttt{<}integer\texttt{>}\ sol \\
Q \equiv \{\forall i : 0 \leq i < v.size : sol[i] = \sum_{j=0}^{j=i} v[j]\}
\end{array}
$$

4.6. Coste de la solución

- El coste de calcular el vector de acumulados es lineal en el número de elementos del vector, que se corresponde con el número de años de los que se han tomado medidas. El algoritmo tiene un bucle que recorre el vector de entrada completo. En cada vuelta del bucle se realiza una asignación, una comparación y dos incrementos, todos ellos de coste constante. Por lo tanto el coste es el número de vueltas del bucle.

- El coste de resolver cada pregunta es constante, ya que se resuelve con dos accesos al vector. Por lo tanto el coste de resolver todas las preguntas es del orden del número de preguntas.

- Como los dos procesos son secuenciales, primero se calcula el vector de acumulados y después se contestan las preguntas. El coste es el máximo entre los dos procesos. Por lo tanto el coste total del problema (incluyendo las preguntas) está en el orden $\mathcal{O}($ max(número de años considerados, número de preguntas realizadas)).

4.7. Errores frecuentes.

- Realizar una implementación de coste cuadrático respecto al número de años considerados y el número de preguntas. Si sumamos el número de nacimientos para cada pregunta que nos hacen, tenemos que el coste es del orden $\mathcal{O}(k * n)$ siendo k el número de preguntas y n el número de años sumados, en el caso peor todos ellos. Como el número de años es un valor $0 < p \leq u \leq 100{,}000$, y el número de preguntas es un valor $0 < m \leq 100{,}000$, tenemos en total unas 10^{10} operaciones, lo que no es ejecutable con un algoritmo cuadrático en un tiempo razonable.

- Los datos de entrada indican que el número de nacimientos de cada año es un entero $0 \leq k < 2^{30}$, Si todos los años nace el máximo de animales, como podemos tener hasta 100.000 años, no es posible almacenar la suma de todos los nacimientos en una variable de tipo `int`. Hay que utilizar el tipo `long long int` para guardar la suma de los valores.

4.8. Implementación en C++.

```
#include <iostream>
#include <fstream>
#include <vector>

using lli = long long int;
```

```
// Calcula un vector con los valores acumulados
// El vector de acumulados tiene una posicion mas que el vector de entrada
// La primera posicion toma el valor cero para facilitar el calculo de las diferencias
void acumular(std::vector<int> const& a, std::vector<lli>& v)
{
    v[0] = 0;
    for (int i = 1; i < v.size(); ++i)
    {
        v[i] = v[i-1] + a[i-1];
    }
}

bool resuelveCaso() {
    // Lee las dos fechas entra las que tenemos datos
    int a1, a2;
    std::cin >> a1 >> a2;
    if (a1 == 0 && a2 == 0) return false;
    // Declara el vector de entrada de datos y lo lee
    int nElem = a2-a1+1;
    std::vector<int> v(nElem);
    for (int i = 0; i < nElem; ++i) {
        std::cin >> v[i];
    }
    // Calcula el vector de acumulados
    std::vector<lli> salida(v.size()+1);
    acumular(v,salida);
    // Lee las preguntas y mira la respuesta en el vector de acumulados
    int m; std::cin >> m;
    for (int i = 0; i < m; ++i) {
        int f1, f2;
        std::cin >> f1 >> f2;
        std::cout << salida[f2-a1+1]-salida[f1-a1] << '\n';
    }
    std::cout << "---\n";
    return true;

}

int main() {
#ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf()); //save old buf and redirect std::cin to casos.txt
#endif

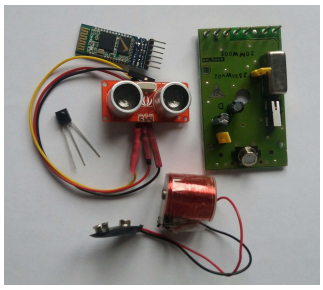
    while (resuelveCaso());

#ifndef DOMJUDGE // para dejar todo como estaba al principio
    std::cin.rdbuf(cinbuf);
    system("PAUSE");
#endif

    return 0;
}
```

5. Desplazar elementos de un vector. Sensores defectuosos.

Sensores defectuosos



Ayer compré unos sensores a un precio increible, sin embargo no funcionan tan bien como yo esperaba. Algunas veces el valor que transmiten es absurdo. He intentado devolverlos, pero me dicen que las ofertas no admiten cambios. Después de mucho estudiar los datos que se producen me he dado cuenta que cuando falla el sensor siempre devuelve el mismo valor erroneo. Siendo así, todavía puedo aprovechar los sensores. Lo único que tengo que hacer es eliminar este valor y quedarme con el resto de los datos.

He hecho un programa que elimina todos los valores erróneos, uno por uno. !!Pero que lento es!!. Al comentárselo a mi hermano me ha explicado una forma de implementar la función mucho más eficiente. Aunque debo de tener cuidado de no variar el orden relativo entre los datos correctos.

Requisitos de implementación.

Debe implementarse una función que reciba un vector con todos los datos tomados por el sensor y el valor del dato erróneo, y lo modifique quitándole todos los datos erróneos.

El coste de la función debe ser del orden del número de datos de entrada.

Entrada

La entrada comienza con un valor entero que indica el número de casos de prueba. Cada caso de prueba consta de dos líneas. En la primera se indica el número de medidas tomadas y el valor erróneo. En la segunda se muestran todos los valores tomados por el sensor.

Las medidas tomadas se encuentran en el rango de valores (-2^{63}... 2^{63}).

Salida

Para cada caso de prueba se escriben dos líneas. En la primera se muestra el número de datos correctos tomados por el sensor, en la segunda los valores correctos.

Entrada de ejemplo

```
4
8 -1
5 -1 -1 10 4 -1 10 7
3 -1
3 5 7
4 0
0 0 0 0
7 0
0 10 -23 0 -12 67 0
```

Salida de ejemplo

```
5
5 10 4 10 7
3
3 5 7
0

4
10 -23 -12 67
```

5.1. Objetivos del problema

- Aprender a eliminar varios elementos de un vector en tiempo lineal respecto al número de elementos del vector, sin utilizar memoria auxiliar.
- Recordar los modificadores de tipo de C++.

5.2. Ideas generales.

- En el problema nos piden eliminar todos los elementos de un vector que cumplen una determinada propiedad. En este caso la propiedad pedida es coincidir con el valor de un parámetro de entrada. El problema se debe resolver recorriendo el vector una única vez y sin utilizar un vector auxiliar.
- Si desplazamos todas las componentes a la derecha del vector por cada elementos que se elimina, el coste de la solución en el caso peor es cuadrático respecto al número de elementos del vector.
- Si se utiliza la función `erase` de la clase `vector` para eliminar los elementos que no cumplen la propiedad, el coste en el caso peor es cuadrático respecto al número de elementos del vector, ya que la función `erase` tiene coste lineal respecto al número de elementos desplazados.
- Para resolver el problema en tiempo lineal respecto al número de elementos del vector y sin utilizar memoria auxiliar, copiaremos en las primeras componentes del vector los elementos considerados correctos, es decir, que no cumplen la propiedad pedida (se eliminan los elementos que cumplen la propiedad). Antes de devolver el control modificaremos la longitud del vector para ajustarlo al número de valores correctos.

5.3. Algunas cuestiones sobre implementación.

- *Modificadores de tipos. Redefinición de un tipo.*

  En la descripción de los datos de entrada al problema, se dice que los elementos del vector están en el rango $(-2^{63} \dots 2^{63})$. Esto significa que los valores no pueden almacenarse en una variable de tipo `int` sino que deben almacenarse en una variable de tipo `long long int`.

  Podemos redefinir el tipo `long long int` dándole un identificador representativo y más fácil de escribir mediante la instrucción:

  ```
  using lli = long long int;
  ```

- *Cómo modificar el tamaño del vector.*

  La función `resize(n)` modifica el tamaño del vector para dejarlo en longitud `n`. Si el nuevo tamaño es menor que el anterior, como ocurre en este problema, la función elimina los últimos elementos del vector para dejar el tamaño pedido. El coste es lineal en el número de elementos que se eliminan, debido al coste de destruir los elementos.

- *Cómo evitar algunas copias de los elementos del vector.*

  Si las componentes del vector son de un tipo básico, copiaremos el valor en la posición `valoresBuenos`, sin modificar el valor de la componente actual. El valor de la componente actual se perderá, pero no importa porque es un valor a eliminar.

  Por el contrario, si las componentes del vector tienen un tipo que implementa la *semántica de movimiento* se utilizará la función `swap` de la STL que permite intercambiar los elementos sin realizar copias.

5.4. Ideas detalladas.

- Debemos utilizar las posiciones del vector cuyos valores son erróneos para colocar los valores correctos. Para ello llevaremos un índice: `valoresBuenos` con la posición del vector que cumple que todas las componentes desde el comienzo del vector hasta este índice son correctos.
  - Si la siguiente componente del vector es igual al elemento que queremos eliminar, se pasa a considerar el siguiente elemento.

- Si la siguiente componente del vector es distinta al valor a eliminar, se copia en la posición indicada por el índice `valoresBuenos` y se avanza el índice `valoresBuenos` para pasar al elemento siguiente.
- La parte del vector entre el índice `valoresBuenos` y el elemento que estemos considerando en esta vuelta del bucle son valores basura que al final del método se desprecian porque quedan a la derecha del índice `valoresBuenos`.

5.5. Especificación e invariante del bucle implementado

Especificación del problema:

1. La precondición indica las propiedades que cumplen los datos de entrada. En nuestro caso el problema tiene como valores de entrada una secuencia de datos que incluyen un valor erróneo y el valor del dato erróneo. En el enunciado se indica únicamente el rango de las medidas tomadas. El valor erróneo se encuentra también en este rango. Al no indicarse nada sobre el número de medidas tomadas, se debe suponer que estas pueden ser cero. Observad que fijamos los valores de entrada a una constante V para poder referirnos a ellos en la postcondición.

2. La cabecera del problema. En ella indicamos el nombre del problema, los datos de entrada o entrada/salida y el tipo del valor calculado. En nuestro caso tenemos la secuencia de valores que representamos en un vector y el valor erróneo. La secuencia de valores de entrada se modifica en la solución del problema. Esto se indica poniendo e/s antes del parámetro. También se hace notar que en la cabecera no se muestra ningún valor de salida, ya que es la propia secuencia de entrada la que se modifica.

3. La postcondición nos muestra las propiedades que cumplen los valores calculados. En nuestro caso, se debe cumplir que ningún valor de la secuencia obtenida como resultado sea erróneo, y que todos los valores correctos de la secuencia de entrada permanezcan en la secuencia al finalizar el programa. También se debe cumplir que el orden relativo de los valores en la secuencia de entrada sea el mismo que el orden relativo de los valores en la secuencia final. La expresión de esta última condición en el lenguaje lógico empleado en la asignatura está fuera del ámbito de la asignatura. Observad que en la postcondición se utiliza v en minúscula para referirnos a la secuencia de valores calculada por el programa y V mayúscula para referirnos a la secuencia de valores de entrada al programa, esto es así porque en la precondición se indica que la secuencia de entrada v tiene el valor V.

4. Observad que en la especificación no se refleja el tipo de datos específico de C++ que se debe utilizar para guardar tanto los datos de entrada como los de salida ya que este es un detalle de implementación. Se indica simplemente que son de tipo entero, utilizaremos *integer* para diferenciarlo del tipo concreto de C++ `int`. En los valores de entrada se debe indicar su rango para que el implementador sepa elegir el tipo adecuado de las variables. En los valores de salida se podría indicar el rango para facilitar la labor del implementador.

$$P \equiv \{0 \leq v.size \wedge v = V \wedge \forall\, i : 0 \leq i < v.size : -2^{63} \leq v[i] < 2^{63} \wedge -2^{63} \leq e \leq 2^{63}\}$$
$$sensores\ (e/svector\texttt{<}integer\texttt{>}\ v, integer\ e)$$
$$Q \equiv \{\forall i : 0 \leq i < v.size : v[i]! = e \wedge \forall i : 0 \leq i < V.size \wedge V[i]! = e : V[i] \in v\}$$

Invariante del bucle que resuelve el problema.

- Un invariante expresa lo que se calcula en el bucle. Por lo tanto un invariante está asociado a un bucle concreto. En nuestra implementación tenemos el siguiente bucle, en el que `v` es el vector de entrada con los datos del sensor y `valorErroneo` el valor que debemos eliminar.

```
for (int k = 0; k < v.size() ;++k) { // Bucle que recorre el vector
    if (v[k] != valorErroneo){ // Si el dato es bueno
        v[valoresBuenos] = v[k];  // lo trasladamos a la zona correcta
        ++valoresBuenos;  // aumentamos la zona correcta
    }
}
```

- No estamos interesados en cualquier invariante del bucle, sino en uno que nos permita probar la corrección del bucle respecto a su precondición y su postcondición. A continuación se muestra los elementos relevantes de la precondición y postcondición del bucle anterior:

  Precondición del bucle: $\{0 \leq V.size \wedge \texttt{valoresBuenos} = 0\}$
  Postcondición del bucle:

  $\{\forall i : 0 \leq i < \texttt{valoresBuenos} : v[i] \neq e \wedge$
  $\forall i : 0 \leq i < v.size \wedge V[i] \neq e : \exists j : 0 \leq j < \texttt{valoresBuenos} : V[i] = v[j]\}$

- La parte en que estamos interesados del invariante es aquella de la que podremos obtener la postcondición. Expresamos en primer lugar que todos los valores de la secuencia hasta el índice que marca la variable `valoresBuenos` deben ser distintos del valor erróneo. Esto se cumple en todas las iteraciones del bucle ya que el valor de la variable `valoresBuenos` sólo se incrementa cuando en esa posición del vector se copia un valor que se ha comprobado que no es erróneo. En la segunda parte del invariante se indica que todos los valores del vector hasta el índice `k`, que es la variable de control del bucle, si no son erróneos están en el vector antes de la posición del índice `valoresBuenos`. esto se cumple en todas las vueltas del bucle, ya que si el valor de la posición `k` no es bueno no se cumple la condición del predicado y si el valor es bueno se copia el valor en la parte *buena* del vector y se aumenta esta parte por lo que existe un índice anterior a `valoresBuenos` que tiene el valor de la componente k-ésima.

  Invariante del bucle:

  $\{\forall i : 0 \leq i < \texttt{valoresBuenos} : v[i] \neq e \wedge$
  $\forall i : 0 \leq i < k \wedge V[i] \neq e : \exists j : 0 \leq j < \texttt{valoresBuenos} : V[i] = v[j]\}$

5.6. Coste de la solución

- En la implementación que se muestra al final del ejercicio y que sigue las ideas presentadas anteriormente, el vector se recorre completo una única vez mediante un bucle `for`. En cada iteración en el caso peor se realizarán una asignación, dos incrementos y dos comparaciones. Por lo tanto, siendo n es el número de medidas tomadas por el sensor, el coste es aproximadamente $5n$ y el orden de complejidad es $\mathcal{O}(n)$.

5.7. Errores frecuentes.

- Implementar una función con coste cuadrático respecto al número de elementos del vector, bien por utilizar la función `erase` o por desplazar todos los elementos a la derecha cuando se encuentra un valor erroneo.

- Implementar una función que devuelve como resultado un vector diferente del de entrada con los datos no eliminados. Esta solución utiliza un vector diferente del vector de entrada y por lo tanto no cumple con los requisitos expuestos en el enunciado del problema.

5.8. Implementación en C++.

```
#include <iostream>
#include <fstream>
#include <vector>

using lli = long long int;

// Funcion que resuelve el problema
void resolver (std::vector<lli> & v, int valorErroneo) {
    int valoresBuenos = 0;
    for (int k = 0; k < v.size() ;++k) { // Bucle que recorre el vector
        if (v[k] != valorErroneo){ // Si el dato es bueno
            v[valoresBuenos] = v[k];  // lo trasladamos a la zona correcta
            ++valoresBuenos;  // aumentamos la zona correcta
        }
```

```
    }
    v.resize(valoresBuenos);  // Dejamos en el vector unicamente los datos correctos
}
// Lectura de los datos de entrada, llamada a la funcion resolver y salida de datos
void resuelveCaso() {
    // Lectura de los datos
    int numElem, valorErroneo;
    std::cin >> numElem >> valorErroneo;
    std::vector<lli> v(numElem);
    for (lli& n : v) std::cin >> n;
    // Resolver el problema
    resolver(v, valorErroneo);
    // Escribir los datos de salida
    if (v.size() > 0) std::cout << v[0];
    for (int i = 1; i < v.size(); ++i)
        std::cout << ' ' << v[i];
    std::cout << '\n';
}

int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif
   // Entrada con numero de casos
    int numCasos;
    std::cin >> numCasos;
    for (int i = 0; i < numCasos; ++i) resuelveCaso()
        ;

    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
   #endif

    return 0;
}
```

6. Segmento máximo, rescate aéreo.

Rescate aéreo 1



Después de sufrir un devastador ataque alienígena, el alto mando ha recibido una llamada de rescate de la ciudad de Nueva York. El grupo de supervivientes se encuentra detrás de la línea de rascacielos de la ciudad. Al otro lado de los edificios se encuentra vigilando una nave extraterrestre. Para evitar que el transporte aéreo encargado del rescate sea detectado por la nave enemiga, hemos encontrado la secuencia más larga de edificios cuya altura es estrictamente mayor que la de nuestro transporte. El alto mando ya ha avisado a los supervivientes para que se dirijan a este punto de encuentro. Suponemos que todos los edificios tienen la misma anchura.

Requisitos de implementación.
El coste de la función debe ser lineal en el número de edificios considerados.

Entrada

La entrada comienza con el número de casos de prueba. Cada caso de prueba tiene dos líneas. En la primera se indica el número de edificios de la línea n, seguido de la altura del transporte t. En la segunda se indica la altura de cada edificio de la línea.

Se supone que la altura del transporte es menor que la altura de alguno de los edificios considerados y por lo menos existe un edificio.

Salida

Para cada caso de prueba se escriben en una línea el comienzo y el final del intervalo. En caso de existir dos intervalos iguales se eligirá el de la izquierda.

Entrada de ejemplo

```
2
10 5
3 6 4 8 9 8 7 2 8 9
7 5
8 8 8 2 3 9 9
```

Salida de ejemplo

```
3 6
0 2
```

6.1. Objetivos del problema

- Aprender el esquema de solución de los problemas que piden calcular el segmento máximo de elementos de un vector que cumplen una propiedad.

6.2. Ideas generales.

- En los problemas de segmento máximo se debe guardar información sobre el segmento más largo que se haya encontrado que cumple la propiedad pedida (*segmento máximo*). Hay tres datos importantes sobre el segmento: su longitud, su punto de comienzo y su punto de final. Solo es necesario guardar dos de estos datos, ya que el tercero se obtiene a partir de los dos guardados. El elegir unos u otros depende del implementador.

- Se debe también guardar información sobre el último segmento que cumple la propiedad pedida (*segmento actual*). Entendemos por segmento actual el que termina en el valor que se está tratando. Sobre este segmento conocemos su final, porque coincide con el valor que estamos tratando, por lo tanto solo es necesario guardar su longitud o su punto de comienzo. Como en el caso anterior uno de estos datos se obtiene a partir de los otros dos.

- El problema se resuelve con un único bucle que recorre el vector. En cada iteración se actualiza el segmento actual dependiendo de si el dato que se trata en la iteración cumple la propiedad o no. Una vez actualizado el segmento actual se comprueba si debe actualizarse el segmento máximo.

6.3. Ideas detalladas.

- *Posibles implementaciones del bucle.*

  Las instrucciones del bucle deben actualizar el valor de todas las variables: longitud máxima hasta este momento, longitud del segmento actual, y posición del inicio del segmento máximo. Si el valor que se está tratando en la iteración cumple la propiedad pedida hay que incrementar la longitud del último segmento y en caso de que no la cumpla hay que iniciar un nuevo segmento. Además, si el último segmento supera al que tenemos guardado como el máximo hasta el momento hay que actualizar el segmento máximo. Observamos que el segmento actual solo puede superar al máximo hasta este momento si se incrementa su longitud. Por lo tanto solo es necesario actualizarlo cuando el valor cumple la propiedad.

```
for (int i = 0; i < v.size(); ++i) {
  if (alturaNave < v[i]) { // elemento bueno
    ++longActual;
    if (longActual > longSegMax) { // mejora el segmento
      longSegMax = longActual;
      iniSegMax = i - longActual + 1;
    }
  }
  else longActual = 0;
}
```

  Otra solución posible consiste en controlar si el segmento actual supera al segmento máximo cuando encontramos un valor que no cumple la propiedad, es decir, cuando termina el segmento válido. En esta solución, es muy importante ver que si el último segmento del vector es válido y es el de tamaño máximo, quedará sin actualizar al acabar el bucle, ya que el segmento terminó con el final del vector en lugar de con un elemento que no cumple la propiedad. Por lo tanto hay que comprobar al terminar el bucle si el último segmento es mayor que el encontrado hasta este momento.

```
for (int i = 0; i < v.size(); ++i) {
  if (alturaNave < v[i])  ++longActual;  // elemento bueno
  else {
    if (longActual > longSegMax) { // mejora el segmento
      longSegMax = longActual;
      iniSegMax = i - longActual + 1;
    }
    longActual = 0;
```

```
  }
}
if (longActual > longSegMax) { // mejora el segmento
  longSegMax = longActual;
  iniSegMax = i - longActual + 1;
}
```

Ambas soluciones tienen coste lineal en el número de elementos del vector. Hay que notar que el caso peor es diferente para cada una de ellas. En la primera solución el caso peor ocurre cuando todos los edificios tienen una altura mayor que la nave. En este caso se modifica el segmento máximo para cada elemento del vector. En la segunda solución el caso peor ocurre cuando la altura de los edificios va alternando, una más alta y a continuación una más baja. Se modifican los valores del segmento máximo cada dos elementos. Por lo tanto la constante multiplicativa es el doble en la primera solución respecto a la segunda. Sin embargo, si el tiempo no es muy crítico se hace notar que es bastante más sencillo probar la corrección de la primera solución que la de la segunda.

6.4. Especificación e invariante del bucle implementado.

Especificación del problema:

1. La precondición indica las propiedades que cumplen los datos de entrada. En nuestro caso el problema tiene como valores de entrada la altura de los edificios y la altura de la nave. El número de edificios no se considera dato de entrada ya que es el número de datos que nos dan sobre la altura de los edificios. El enunciado indica que al menos existe un edificio cuya altura es mayor que la de la nave de rescate. Este dato es importante porque asegura que siempre existe una solución.

2. La cabecera de la función. En ella indicamos el nombre del problema, los datos de entrada o entrada/salida y el tipo del valor calculado. En nuestro caso tenemos dos valores de entrada: la secuencia de alturas de los edificios que representamos en un vector y la altura de la nave. Como salida obtenemos la posición en la secuencia del principio y final del segmento de longitud máxima cuyas componentes tienen todas un valor mayor que la altura de la nave, que representamos mediante el par [`ini fin`]. Observad que se proporciona un identificador tanto a los valores de entrada como al valor de salida para poder referirnos a ellos en la precondición y en la postcondición.

3. La postcondición nos muestra las propiedades que cumplen los valores calculados. En nuestro caso, se obtienen dos valores que representan el principio y final del intervalo. La primera propiedad de la postcondición expresa que el segmento es máximo. Cualquier otro segmento que cumpla la propiedad tiene longitud menor o igual que el segmento [ini..fin]. La segunda propiedad expresa que el segmento máximo es el que se encuentra más a la izquierda de la secuencia. Se indica que cualquier otro segmento máximo encontrado de la misma longitud que el segmento solución comienza en la misma posición que el segmento solución o más a la derecha. Esto significa que no puede comenzar más a la izquierda y por lo tanto el segmento dado como solución es el que se encuentra más a la izquierda. Por último se pide que los índices encontrados pertenezcan a la secuencia.

$$
\begin{array}{l}
P \equiv \{1 \leq v.size \wedge \exists\ i : 0 \leq i < v.size : v[i] > h\} \\
\ rescateAereo\ (vector\texttt{<}integer\texttt{>}\ v, integer\ h)\ dev\ [integer\ ini, integer\ fin] \\
Q \equiv \{fin - ini = max\ i,j : 0 <= i <= j < v.size \wedge mayores(v,i,j,h) : j - i \wedge \\
\qquad \forall\ i,j : 0 <= i <= j < v.size \wedge mayores(v,i,j,h) \wedge j - i = fin - ini : ini <= i \wedge \\
\qquad 0 \leq ini \leq fin < v.size\}
\end{array}
$$

donde: $mayores(v,i,j,h) \equiv \forall x : i \leq x \leq j : v[x] > h$

Invariante del bucle implementado.

- El invariante se refiere a un bucle concreto. En nuestro caso tenemos el siguiente bucle, donde: `v` es el vector con los datos de las alturas de los edificios; `alturaNave` es la altura de la nave;

`longSegMax` guarda la longitud del segmento máximo; `iniSegMax` guarda el inicio del segmento máximo y `longActual` guarda la longitud del segmento que termina en el elemento que se está tratando.

```
    for (int i = 0; i < v.size(); ++i) {
        if (alturaNave < v[i]) { // elemento bueno
            ++longActual;
            if (longActual > longSegMax) { // mejora el segmento
                longSegMax = longActual;
                iniSegMax = i - longActual + 1;
            }
        }
        else longActual = 0;
    }
}
```

- Se debe conocer la precondición del bucle y la postcondición del bucle. En nuestro caso la parte relevante de la precondición del bucle incluye la precondición del algoritmo y la inicialización de las variables que guardan los valores calculados por el bucle:

  Precondición del bucle:

  $\{1 \leq \texttt{v.size}() \wedge \exists\, i : 0 \leq i < \texttt{v.size}() : v[i] > \texttt{alturaNave} \wedge \texttt{longSegMax} = 0$
  $\wedge \texttt{iniSegMax} = 0 \wedge \texttt{longActual} = 0\}$

  En nuestro problema, el bucle calcula el índice de comienzo del segmento y la longitud de éste, en lugar de los índices de comienzo y final del segmento que pide el problema. Por lo tanto, la postcondición del bucle debe referirse a la posición de inicio y a la longitud del segmento. El final del segmento se calcula en una instrucción posterior al bucle y por lo tanto no forma parte de la postcondición del bucle.

  Postcondición del bucle:

  $\{\texttt{longSegMax} = max\ i, j : 0 \leq i \leq j < \texttt{v.size}() \wedge mayores(v, i, j, \texttt{alturaNave}) : j - i \wedge$
  $\forall\, i, j : 0 \leq i \leq j < \texttt{v.size}() \wedge mayores(v, i, j, \texttt{alturaNave}) \wedge j - i = \texttt{longSegMax} : \texttt{iniSegMax} \leq i \wedge$
  $0 \leq \texttt{iniSegMax} < \texttt{v.size}() \wedge 1 \leq \texttt{longSegMax} < \texttt{v.size}()\}$

- El invariante debe expresar lo que calcula el bucle, teniendo en cuenta que sólo puede expresar lo calculado hasta la variable de control del bucle.

  Cómo la variable que controla el bucle tiene identificador `i`, renombramos las variables ligadas de los predicados de la postcondición del bucle para que no aparezca esta variable como variable ligada a un predicado.

  Además el invariante tiene que expresar los valores que guardan las variables que aparecen en el bucle aunque no formen parte de la postcondición de éste. En concreto debe indicar las propiedades que cumple el valor que guarda la variable `longActual`. Estas se muestran en la última propiedad del invariante.

  Invariante:

$\{\texttt{longSegMax} = max\ k_1, k_2 : 0 \leq k_1 \leq k_2 < i \wedge mayores(v, k_1, k_2, \texttt{alturaNave}) : k_2 - k_1 \wedge$
$\forall\, k_1, k_2 : 0 \leq k_1 \leq k_2 < i \wedge mayores(v, k_1, k_2, \texttt{alturaNave}) \wedge k_2 - k_1 = \texttt{longSegMax} : \texttt{iniSegMax} \leq k_1 \wedge$
$0 \leq \texttt{iniSegMax} < i \wedge 1 \leq \texttt{longSegMax} < i \wedge$
$\texttt{longActual} = min\ k_1 : 0 \leq k_1 < i \wedge mayores(v, k_1, i - 1, \texttt{alturaNave}) : k_1\}$

6.5. Coste de la solución

Vemos el coste de la solución que se muestra al final del problema. En ella se utiliza la primera solución explicada en el apartado anterior. El bucle recorre todos los elementos del vector. En cada iteración del bucle se consultan un elemento del vector y se hacen dos incrementos, dos comparaciones y tres asignaciones, todas ellas operaciones de coste constante. Por lo tanto cada iteración tiene coste constante. Como el bucle hace n iteraciones, siendo n el número de edificios, el coste del bucle es del orden de $\mathcal{O}(n)$.

6.6. Errores frecuentes.

- Proponer un algoritmo de coste cuadrático respecto al número de elementos.
- En la segunda solución, olvidarse de comprobar el último segmento después del bucle.

6.7. Modificaciones al problema.

- *Obtener los segmentos con longitud máxima.*

  Nos pueden pedir obtener el segmento máximo más a la derecha del vector, o más a la izquierda, u obtenerlos todos. En los dos primeros casos es necesario utilizar una variable en la que se guarda el comienzo o final del segmento máximo encontrado hasta este momento. En el caso en que nos pidan obtener todos los segmentos máximos es necesario utilizar un vector para almacenar el comienzo o final de todos los segmentos máximos encontrados.

  El inicio/final del segmento máximo se actualiza cada vez que se encuentra un segmento mayor. Si nos piden el segmento máximo más a la izquierda debemos actualizar el inicio/final cuando encontramos un intervalo estrictamente mayor, si nos piden el segmento más a la derecha actualizamos el inicio/final cuando encontramos un intervalo mayor o igual, y en caso de que nos pidan todos los comienzos actualizaremos el vector cuando se encuentre un segmento estrictamente mayor y añadiremos un nuevo inicio/final cuando se encuentre un intervalo igual.

- *Obtener el segmento de suma máxima.*

  En este caso llevaremos en una variable la suma máxima encontrada hasta el momento en que estamos ejecutando y en otra la suma del último segmento que estamos tratando. El segmento actual acaba si la suma se hace negativa ya que en este caso conviene comenzar un nuevo segmento en lugar de acumular al segmento anterior.

6.8. Implementación en C++.

```
#include <iostream>
#include <fstream>
#include <vector>

// Devuelve la posicion inicial y final del segmento maximo
std::pair<int, int> resolver(std::vector<int> const& v, int alturaNave) {
    int longSegMax = 0, iniSegMax = 0, longActual = 0;
    for (int i = 0; i < v.size(); ++i) {
        if (alturaNave < v[i]) { // elemento bueno
            ++longActual;
            if (longActual > longSegMax) { // mejora el segmento
                longSegMax = longActual;
                iniSegMax = i - longActual + 1;
            }
        }
        else longActual = 0;
    }
    return {iniSegMax, iniSegMax + longSegMax - 1};
}

void resuelveCaso() {
    // Lectura de los datos
    int numElem, alturaNave;
    std::cin >> numElem >> alturaNave;
    std::vector<int> v(numElem);
    for (int& i : v) std::cin >> i;
    // Resolver el problema
    std::pair<int, int> sol = resolver(v, alturaNave);
    // escribir el resultado
    std::cout << sol.first << ' ' << sol.second << "\n";
```

```
}

int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif

    // Entrada por numero de casos
     int numCasos;
    std::cin >> numCasos;
    for (int i = 0; i < numCasos; ++i) {
        resuelveCaso();
    }

    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
   #endif

    return 0;
}
```

7. Mezcla ordenada. Un móvil nuevo

Un móvil nuevo



Queremos cambiar de móvil y hemos buscado los precios de los teléfonos que ofrecen un par de marcas. Tenemos una lista con los precios que figuran en el catálogo de la marca *A* y otra lista con los precios de los teléfonos de la marca *B*. Ambas listas están ordenadas por precio de menor a mayor y en la misma lista no hay precios repetidos.

Para hacernos una idea de la variedad de precios que hay, queremos obtener una lista con todos los precios de una u otra lista ordenados de menor a mayor precio. Si un precio figura en las dos listas lo pondremos una única vez en nuestra lista final.

Requisitos de implementación.

El problema debe resolverse con una función que recibe dos vectores con los precios que ofrecen las dos marcas de teléfonos. El resultado de la función es otro vector con los precios de ambas listas ordenados.

La función debe tener un coste lineal en el número de elementos de las dos listas. Para ello solo se puede acceder a cada dato del vector una vez.

Entrada

La entrada comienza con un valor entero que indica el número de casos de prueba. Cada caso de prueba consta de tres líneas. La primera línea tiene dos valores que indican el número de precios de cada una de las listas. Las dos líneas siguientes tienen los precios de cada una de las listas.

El número de precios de cada lista es mayor o igual que cero y menor que 100.000. Los precios son enteros positivos.

Salida

Para cada caso de prueba se escribe en una línea la lista de precios resultante ordenada de menor a mayor y sin repetidos.

Entrada de ejemplo

```
3
4 5
3 6 10 20
1 2 10 15 20
0 2

5 10
3 3
1 2 3
4 5 6
```

Salida de ejemplo

```
1 2 3 6 10 15 20
5 10
1 2 3 4 5 6
```

7.1. Objetivos del problema

- Practicar el algoritmo de la mezcla ordenada sin elementos repetidos.
- Estudiar las modificaciones al algoritmo si se consideran elementos repetidos en los vectores.

7.2. Ideas generales.

- Los vectores de entrada están ordenados.
- Se recorren ambos vectores de izquierda a derecha, comparando en cada iteración un valor del primer vector con un valor del segundo vector, y avanzando en el vector cuya componente es menor.
- Cuando se termina de recorrer uno de los dos vectores y ya no se pueden comparar los valores de uno con los del otro, se vuelcan los valores del vector que no se ha acabado de recorrer en el vector de salida.

7.3. Algunas cuestiones sobre implementación.

- *Paso de parámetros.*
  - La función recibe el vector de datos y no lo modifica, por lo tanto el parámetro es `const&`.
- *Valor de retorno de la función*
  - La función devuelve un valor de tipo `vector`. Se puede devolver como resultado de la función o como parámetro por referencia.
- La mezcla siempre se realiza en un vector diferente de los vectores dados. Es conveniente declarar el vector vacío y utilizar el método `push_back` de la clase vector para añadir los elementos a la solución. No se puede declarar un vector con tamaño, ya que se desconoce el número de elementos que coincidirán en los dos vectores.
- El condicional que controla el bucle `while`, comprueba si se cumple la condición antes de comenzar el bucle. No es necesario escribir una instrucción condicional antes del bucle para comprobar dicha condición.

```
while (i < v1.size()) {
   sol.push_back(v1[i]);
   ++i;
}
```

  No es necesaria una instrucción condicional que compruebe la condición `i < v.size()` antes del bucle.

7.4. Ideas detalladas.

- Se utilizan dos índices para recorrer ambos vectores de izquierda a derecha. Los índices se inicializan a la primera componente.
- El primer bucle se ejecuta mientras no se acaba de recorrer alguno de los vectores.
  - Si es menor el valor del primer vector, se copia el valor en el vector de salida y se avanza el índice del primer vector para comparar la siguiente componente en la próxima iteración.
  - Si es menor el valor del segundo vector, se copia este valor en el vector de salida y se avanza el índice de este vector.
  - Si los valores son iguales, como el enunciado indica que no se deben tener valores repetidos, se guarda el valor en el vector de salida y se avanza en los dos índices. El enunciado indica que los vectores de entrada no tienen valores repetidos, por lo tanto el vector de salida no los tendrá.

- Una vez acabado el bucle principal, se implementan dos bucles secuenciales. En el primero se vuelcan los valores restantes del primer vector en el vector de salida y en el segundo se vuelcan los valores restantes del segundo vector al vector de salida. De estos dos bucles solamente uno dará vueltas. El que corresponda con el vector al que le quedan valores.

7.5. Especificación e invariante del bucle implementado.

Especificación del problema:

1. La precondición indica las propiedades que cumplen los datos de entrada. En nuestro caso el problema tiene como valores de entrada dos vectores de números enteros ordenados de menor a mayor. Esto se indica en la precondición mediante el predicado *ord* definido más abajo. Se indica también que en la misma lista no hay valores repetidos, aunque pueden repetirse valores en diferente listas. Observad que en el predicado *ord* los valores se pide que estén ordenados estrictamente ($<$), esto significa que no puede haber dos valores iguales en una lista. En cuanto a que los valores se puedan repetir en listas diferentes, no es necesario indicarlo en la especificación, ya que por defecto se pueden repetir. Habría que indicarlo si tuviesen que ser diferentes. Las listas pueden ser vacías y pueden tener un número diferente de elementos cada una de ellas. Lo primero se indica especificando que el tamaño de la lista es mayor o igual que cero. La segunda condición se da por supuesta si no se dice nada. Sobre los valores que componen en vector se indica que son números enteros.

2. La cabecera de la función. En ella indicamos el nombre del problema, los datos de entrada o entrada/salida y el tipo del valor calculado. En nuestro caso tenemos las dos secuencias de valores ordenados como datos de entrada y la secuencia con la mezcla como salida. Observad que se proporciona un identificador tanto a los valores de entrada como al valor de salida para poder referirnos a ellos en la precondición y en la postcondición. En la cabecera se indica que la función devuelve un vector con la mezcla ordenada. Al implementar la función, por razones de eficiencia, se debe utilizar un parámetro por referencia en lugar de devolver el vector como resultado de la función.

3. La postcondición nos muestra las propiedades que cumplen los valores calculados. En nuestro caso, se obtiene un vector con la mezcla ordenada. La primera propiedad de la postcondición expresa que los valores del vector solución están ordenados de menor a mayor. La segunda propiedad expresa que todos los valores del primer vector de entrada se encuentran en el vector de salida y la tercera propiedad expresa que los valores del segundo vector de entrada se encuentran en el vector de salida. La cuarta propiedad indica que en el vector de salida no hay ningún elemento que no se encuentre en alguno de los vectores de entrada. Como en el caso de los valores de entrada, el vector de salida no tiene elementos repetidos porque el orden especificado entre ellos es estricto.

$$
\begin{array}{l}
P : \{0 \leq v1.size \wedge 0 \leq v2.size \wedge ord(v1) \wedge ord(v2) \wedge \\
\quad \forall i : 0 \leq i < v1.size : 0 \leq v1[i] < 2^{31} \wedge \forall i : 0 \leq i < v2.size : 0 \leq v2[i] < 2^{31}\} \\
\ mezcla\ (vector\texttt{<}integer\texttt{>}\ v1, vector\texttt{<}integer\texttt{>}\ v2) dev\ vector\texttt{<}integer\texttt{>}\ v \\
Q : \{\forall u :: 0 \leq u < v.size - 1 : v[u] < v[u+1] \wedge \\
\quad \forall k :: 0 \leq k < v1.size : in(v1[k], v) \wedge \\
\quad \forall k :: 0 \leq k < v2.size : in(v2[k], v) \wedge \\
\quad \forall k : 0 \leq kv.size : in(v[k], v1) \vee in(v[k], v2)\}
\end{array}
$$

donde:

$ord(v) \equiv \forall u, w :: 0 \leq u < w < v.size : v[u] < v[w]$

$in(x, v) \equiv \exists k : 0 \leq k < v.size : v[k] = x$

Invariante del primer bucle implementado.

- El invariante se refiere a un bucle concreto. Nuestro programa utiliza tres bucles en secuencia. Cada uno de ellos tiene un invariante diferente.

- En el primer bucle, se recorren los vectores de entrada hasta que uno de ellos finaliza. `v1` y `v2` son los vectores con los datos de entrada; `sol` es el vector con los valores de salida; e `i`, `j` son los índices que recorren cada uno de los vectores de entrada..

```
while (i < v1.size() && j < v2.size()) {
    if (v1[i] < v2[j]) { // Elemento del primer vector menor
        sol.push_back(v1[i]);
        ++i;
    }
    else if (v1[i] > v2[j]) { // Elemento del segundo vector menor
        sol.push_back(v2[j]) ;
        ++j;
    }
    else { // valores iguales
        sol.push_back(v1[i]) ;
        ++i; ++j;
    }
}
```

- Se debe conocer la precondición del bucle y la postcondición del bucle. En nuestro caso la parte relevante de la precondición del bucle incluye la precondición del algoritmo y la inicialización de las variables que guardan los valores calculados por el bucle:

  Precondición del primer bucle:

  $$\{0 \leq v1.size \wedge 0 \leq v2.size \wedge ord(v1) \wedge ord(v2) \wedge i = 0 \wedge j = 0\}$$

  La postcondición expresa que los valores del vector solución son la mezcla ordenada de los vectores de entrada hasta el valor que indican los índices que recorren los vectores de entrada.

  Postcondición del primer bucle:

  $$\begin{array}{l}\{\forall u :: 0 \leq u < sol.size - 1 : sol[u] < sol[u+1] \wedge \\ \forall k :: 0 \leq k < i : in(v1[k], sol) \wedge \\ \forall k :: 0 \leq k < j : in(v2[k], sol) \wedge \\ \forall k : 0 \leq ksol.size : in(sol[k], v1) \vee in(sol[k], v2)\}\end{array}$$

- El invariante debe expresar lo que calcula el bucle, teniendo en cuenta que sólo puede expresar lo calculado hasta la variable de control del bucle.

  En este caso hay dos variable que controlan el bucle `i` y `j`.

  Invariante del primer bucle:

  $$\begin{array}{l}\{\forall u :: 0 \leq u < sol.size - 1 : sol[u] < sol[u+1] \wedge \\ \forall k :: 0 \leq k < i : in(v1[k], sol) \wedge \\ \forall k :: 0 \leq k < j : in(v2[k], sol) \wedge \\ \forall k : 0 \leq ksol.size : in(sol[k], v1) \vee in(sol[k], v2)\}\end{array}$$

  Vemos que en este caso el invariante coincide con la postcondición del bucle.

Invariante del segundo bucle implementado.

- En el segundo bucle, se recorre el vector `v1` desde el valor del índice `i` hasta el final.

```
while (i < v1.size()) {
    sol.push_back(v1[i]);
    ++i;
}
```

- Se debe conocer la precondición del bucle y la postcondición del bucle. En nuestro caso la precondición del bucle es la postcondición del bucle anterior, ya que los dos bucles son secuenciales y no hay ninguna instrucción entre ellos:

  Precondición del segundo bucle:

$$\{\forall u :: 0 \leq u < sol.size - 1 : sol[u] < sol[u+1] \wedge \\ \forall k :: 0 \leq k < i : in(v1[k], sol) \wedge \\ \forall k :: 0 \leq k < j : in(v2[k], sol) \wedge \\ \forall k : 0 \leq ksol.size : in(sol[k], v1) \vee in(sol[k], v2)\}$$

  La postcondición expresa que los valores del vector solución son la mezcla ordenada de los vectores de entrada hasta el final del vector `v1` y hasta el valor que indique el índice `j` en el vector `v2`..

  Postcondición del bucle:

$$\{\forall u :: 0 \leq u < sol.size - 1 : sol[u] < sol[u+1] \wedge \\ \forall k :: 0 \leq k < v1.size : in(v1[k], sol) \wedge \\ \forall k :: 0 \leq k < j : in(v2[k], sol) \wedge \\ \forall k : 0 \leq ksol.size : in(sol[k], v1) \vee in(sol[k], v2)\}$$

- El invariante debe expresar lo que calcula el bucle, teniendo en cuenta que sólo puede expresar lo calculado hasta la variable de control del bucle.

  Invariante:

$$\{\forall u :: 0 \leq u < sol.size - 1 : sol[u] < sol[u+1] \wedge \\ \forall k :: 0 \leq k < i : in(v1[k], sol) \wedge \\ \forall k :: 0 \leq k < j : in(v2[k], sol) \wedge \\ \forall k : 0 \leq ksol.size : in(sol[k], v1) \vee in(sol[k], v2)\}$$

  En este caso el invariante no coincide con la postcondición del bucle. En la segunda propiedad el invariante expresa que se cumple hasta el valor de la variable `i`, mientras que la postcondición expresa que se cumple para el vector completo.

El tercer bucle es similar al segundo, pero terminando de recorrer el vector `v2`.

7.6. Coste de la solución

- El programa tiene tres bucles en secuencia. Las instrucciones de todos ellos tienen coste constante porque son asignaciones, expresiones aritméticas y relacionales y la operación que añade un elemento al final del vector que es de coste constante.
- El número de vueltas que da el primer bucle es el número de componentes del vector más *corto* más aquellas componentes del vector más *largo* cuyos valores son menores que los valores del vector *corto*.
- De los siguientes bucles solo se ejecuta uno de ellos y el número de vueltas coincide con el número de elementos del vector que no se recorrieron en el primer bucle.
- Por lo tanto entre los tres bucles se recorren todas las componentes de los dos vectores una sola vez y el coste de cada iteración es constante.
- Por lo tanto, el coste es del orden del número de precios de las dos listas.

7.7. Errores frecuentes.

- Realizar una copia de los vectores en el paso de parámetros a la función.
- No utilizar el esquema de la mezcla ordenada e intentar resolver el problema con un solo bucle.
- Escribir una condición de parada del primer bucle (que termine uno de los dos vectores) incorrecta. El bucle debe ejecutarse mientras haya elementos que recorrer en los dos vectores.

7.8. Modificaciones al problema.

- *Realizar la mezcla ordenada de vectores que pueden tener elementos repetidos, obteniéndose todas las repeticiones en el resultado.*

  El problema se resuelve utilizando el mismo esquema que la mezcla ordenada sin repeticiones. Se modifican las acciones que se realizan cuando se comparan los dos vectores. Se puede resolver de dos formas:

  - Escribir un valor en cada iteración. Cuando los valores coinciden es indiferente la rama del condicional en que se escriba, se debe optar por una de ellas.
  - Escribir los dos valores cuando coinciden y avanzar en los dos índices.

  Ambas soluciones tienen el mismo coste.

7.9. Implementación en C++.

```
#include <iostream>
#include <fstream>
#include <vector>

std::vector<int> resolver(std::vector<int>const& v1, std::vector<int>const& v2) {
    std::vector<int> sol;
    // Bucle que recorre los dos vectores mientras no termina ninguno
    int i = 0; int j = 0;
    while (i < v1.size() && j < v2.size()) {
        if (v1[i] < v2[j]) { // Elemento del primer vector menor
            sol.push_back(v1[i]);
            ++i;
        }
        else if (v1[i] > v2[j]) { // Elemento del segundo vector menor
            sol.push_back(v2[j]) ;
            ++j;
        }
        else { // valores iguales
            sol.push_back(v1[i]) ;
            ++i; ++j;
        }
    }
    // Quedan elementos en el primer vector
    while (i < v1.size()) {
        sol.push_back(v1[i]);
        ++i;
    }
    // Quedan elementos en el segundo
    while (j < v2.size()) {
        sol.push_back(v2[j]);
        ++j;
    }
    return sol;
}
```

```
void resuelveCaso() {
    // lectura de los datos
    int n1, n2;
    std::cin >> n1 >> n2;
    std::vector<int> v1(n1);
    std::vector<int> v2(n2);
    for (int& i : v1) std::cin >> i;
    for (int & j : v2) std::cin >> j;
    // Resolver el problema
    std::vector<int> sol = resolver(v1, v2);
    // Escribir la solucion
    if (!sol.empty()) {
        std::cout << sol[0];
        for (int i = 1; i < sol.size(); ++i)
            std::cout << ' ' << sol[i];
    }
    std::cout << '\n';
}
int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif
   // Entrada con numero de casos
    int numCasos;
    std::cin >> numCasos;
    for (int i = 0; i < numCasos; ++i) resuelveCaso()
        ;

    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
   #endif

    return 0;
}
```

8. Cálculo de la moda. Juego de dados

Juego de dados



Dos amigos han pasado la tarde jugando a los dados. El juego consiste en meter dos dados en un cubilete, luego lanzar los dados y ver cuál de los dos ha conseguido una puntuación mayor. Cada uno ha llevado la cuenta de todas sus tiradas, apuntando el valor de los dados. Ahora quieren saber cuál es el valor que más veces ha obtenido cada uno.

Requisitos de implementación.

El problema debe resolverse con una función que recibe en un vector los datos de entrada y devuelve un vector con los valores que se repiten el número máximo de veces.

Se utilizará un vector auxiliar para contar las veces que aparece cada número.

Entrada

La entrada consta de una serie de casos de prueba. Cada caso consta de dos líneas, en la primera se muestra el número de veces que se han tirado los dados y el valor máximo que se puede conseguir con las tiradas (hay que tener en cuenta que no todos los dados tienen 6 caras, hay dados con muchas caras, con los que se pueden obtener más de 6 valores diferentes), y en la segunda se muestran los valores obtenidos con los dados. El final de casos se marca con el valor -1.

Los valores de los dados comienzan en uno. El máximo número de caras de un dado es 10.000. El número de veces que se tiran los dados es mayor o igual que uno y menor o igual que 200.000.

Salida

Para cada caso de prueba se escribe en una línea el valor de los dados que más se ha repetido en la secuencia. Si hay varios que se repiten el número máximo de veces se muestran todos ellos, ordenados de mayor a menor.

Entrada de ejemplo

```
7 12
5 3 8 5 10 10 5
6 10
5 9 9 2 5 2
1 24
12
-1
```

Salida de ejemplo

```
5
9 5 2
12
```

8.1. Objetivos del problema

- Conocer las diferentes formas de calcular la moda de una colección de elementos. Se conoce como moda al elemento que más veces se repite en la colección.
- Aprender a acumular la información en una estructura de datos (vector) para resolver el problema de forma eficiente.

8.2. Ideas generales.

- El problema pide calcular la moda de una colección de valores. Los valores están acotados y por lo tanto se puede utilizar un vector auxiliar para contar las apariciones de cada elemento.
- Se declara un vector auxiliar con tantas componentes como valores posibles haya en el vector. Si no se conoce el mínimo y el máximo de los valores del vector, se calculará. En el vector se almacena el número de veces que aparece cada valor.
- Observad que si el número de datos diferentes es pequeño pero están muy dispersos puede utilizarse un TAD diccionario que se estudiará en la asignatura de ED.

8.3. Algunas cuestiones sobre implementación.

- El problema pide que se escriban todos los valores que aparecen un número máximo de veces. Estos valores los guardaremos en un vector que será el resultado de la función como pide el enunciado.
- Lo más sencillo es implementar dos bucles. En el primero se calcula el valor máximo y en el segundo se recorre el vector guardando todas aquellas componentes cuyo valor sea igual al máximo.

8.4. Ideas detalladas.

- Se declara un vector auxiliar en el que se cuenta el número de veces que aparece cada valor.
- El tamaño del vector auxiliar es el máximo rango de los valores de los dos vectores de entrada. En este problema los valores están en el rango *[0..maxValor]*, siendo *maxValor* un dato de entrada.
- En general, buscaremos el valor máximo y mínimo de los datos y declararemos el vector auxiliar de tamaño *maximo - minimo +1*. La información del valor i-ésimo se encontrará en la componente *i-minimo* del vector.
- Se recorre el vector de entrada y se incrementa la componente del vector auxiliar correspondiente al valor del vector de entrada.
- Una vez creado el vector auxiliar lo recorremos buscando el valor máximo.
- Como en este problema nos piden que devolvamos todos los valores que tiene un número de apariciones máximo, haremos un nuevo bucle que recorra el vector de apariciones y guarde todos los índices cuyas componentes tengan el valor máximo en un vector que devolveremos como resultado de la función.

8.5. Especificación e invariante del bucle implementado.

Especificación del problema:

1. La precondición indica las propiedades que cumplen los datos de entrada. En nuestro caso el problema tiene como entrada los valores de las tiradas de los dados. A parte de los límites impuestos por el enunciado a estos valores, lo más representativo es que al menos existe una tirada.
2. La cabecera de la función. En ella indicamos el nombre del problema, los datos de entrada o entrada/salida y el tipo del valor calculado. En nuestro caso tenemos como entrada la secuencia de tirada de dados y como salida un vector con los valores que se han repetido más veces. Se observa que diferentes valores se pueden repetir el máximo de veces. Por ejemplo 2 2 3 3, cada valor se repite dos veces, por lo tanto los dos valores son moda.

3. La postcondición nos muestra las propiedades que cumplen los valores calculados. En nuestro caso, se debe indicar que todos los valores que aparecen en el vector de salida se repiten el número máximo de veces. Para contar el número de veces que se repite un valor se utiliza la función lógica #. Para indicar que se repite el máximo número de veces, se expresa que todos los demás valores se repiten igual o menos veces.

$$
\begin{array}{l}
P : \{0 < v.size\} \\
\ dados\ (vector\texttt{<}integer\texttt{>}\ v) dev\ vector\texttt{<}integer\texttt{>}\ sol \\
Q : \{\forall x : 0 \leq x < sol.size : (\forall k : 0 \leq k < v.size : ig(v, v[k]) \leq ig(v, sol[x]))\}
\end{array}
$$

donde:

$ig(v, m) \equiv \#x : 0 \leq x < v.size : m == v[x]$

Invariante del bucle implementado.

- El invariante se refiere a un bucle concreto. Nuestro programa utiliza tres bucles en secuencia. Cada uno de ellos tiene un invariante diferente.
- En el primer bucle, se recorren los vectores de entrada hasta que uno de ellos finaliza. `v1` y `v2` son los vectores con los datos de entrada; `sol` es el vector con los valores de salida; e `i`, `j` son los índices que recorren cada uno de los vectores de entrada..

```
while (i < v1.size() && j < v2.size()) {
    if (v1[i] < v2[j]) { // Elemento del primer vector menor
        sol.push_back(v1[i]);
        ++i;
    }
    else if (v1[i] > v2[j]) { // Elemento del segundo vector menor
        sol.push_back(v2[j]) ;
        ++j;
    }
    else { // valores iguales
        sol.push_back(v1[i]) ;
        ++i; ++j;
    }
}
```

- Se debe conocer la precondición del bucle y la postcondición del bucle. En nuestro caso la parte relevante de la precondición del bucle incluye la precondición del algoritmo y la inicialización de las variables que guardan los valores calculados por el bucle:

  Precondición del primer bucle:

  $\{0 \leq v1.size \wedge 0 \leq v2.size \wedge ord(v1) \wedge ord(v2) \wedge i = 0 \wedge j = 0\}$

  La postcondición expresa que los valores del vector solución son la mezcla ordenada de los vectores de entrada hasta el valor que indican los índices que recorren los vectores de entrada.

  Postcondición del primer bucle:

$$
\begin{array}{l}
\{\forall u :: 0 \leq u < sol.size - 1 : sol[u] < sol[u+1] \wedge \\
\ \forall k :: 0 \leq k < i : in(v1[k], sol) \wedge \\
\ \forall k :: 0 \leq k < j : in(v2[k], sol) \wedge \\
\ \forall k : 0 \leq ksol.size : in(sol[k], v1) \vee in(sol[k], v2)\}
\end{array}
$$

- El invariante debe expresar lo que calcula el bucle, teniendo en cuenta que sólo puede expresar lo calculado hasta la variable de control del bucle.

  En este caso hay dos variable que controlan el bucle `i` y `j`.

Invariante del primer bucle:

$$\{\forall u :: 0 \leq u < sol.size - 1 : sol[u] < sol[u+1] \wedge \\ \forall k :: 0 \leq k < i : in(v1[k], sol) \wedge \\ \forall k :: 0 \leq k < j : in(v2[k], sol) \wedge \\ \forall k : 0 \leq ksol.size : in(sol[k], v1) \vee in(sol[k], v2)\}$$

Vemos que en este caso el invariante coincide con la postcondición del bucle.

8.6. Coste de la solución

- En la solución del problema hay dos partes.
- Primero se crea el vector auxiliar con las apariciones de cada elemento. Se implementa con un bucle que da un número de vueltas igual al número de tiradas de dados que se han realizado. En cada vuelta se modifica una componente de un vector, por lo tanto el coste de cada vuelta es constante. El coste de esta primera parte es lineal en el número de tiradas de dados.
- Después se recorre el vector auxiliar para encontrar el valor máximo y contar el número de veces que aparece. Se hace con dos bucles que tienen el mismo coste. El número de vueltas que da el bucle es el número de posibles valores que se pueden obtener con los dados. El coste de cada vuelta es constante. Por lo tanto el coste de la segunda parte es del orden del número de valores posibles de los dos dados.
- Por lo tanto el coste del algoritmo es el máximo entre el número de tiradas de los dados realizadas y el número de posibles valores que puedan tomar los dados.

8.7. Errores frecuentes.

- No tener en cuenta que el valor de los dados pertenece al intervalo [1 .. maxValor]. El índice del vector en el que se cuentan las apariciones de cada valor debe ser el valor menos uno. Esto también se debe tener en cuenta cuando se crea el vector solución con los valores que mas se han repetido, ya que los índices del vector en que se cuentan las apariciones serán una unidad menor que el valor. Otra posibilidad es declarar el vector donde se cuentan las apariciones de tamaño una unidad mas que el máximo de los valores y no utilizar la componente cero.
- La salida se pide ordenada de mayor a menor. Para obtenerla así directamente, el vector de apariciones en el que se han contado las apariciones de cada elemento se debe recorrer de derecha a izquierda. De esta forma el primer elemento del vector de salida será el mayor índice.

8.8. Implementación en C++.

```
#include <iostream>
#include <fstream>
#include <vector>

// Devuelve un vector con los valores que se repiten el numero maximo de veces
std::vector<int> resolver(std::vector<int> const& v, int maximoValores){
    // Cuenta el numero de apariciones
    std::vector<int> apariciones(maximoValores);
    for (int n : v) ++apariciones[n-1]; // Los valores de v comienzan en 1
    // Calcula el maximo del vector apariciones
    int aparicionesMax = apariciones[0];
    for (int n : apariciones) aparicionesMax = std::max(aparicionesMax, n);
    // Anade a sol los valores que aparecen aparicionesMax veces
    std::vector<int>sol;
    for (int i = apariciones.size(); i > 0 ; --i)
        // se suma 1 porque los valores de v comenzaban en 1.
```

```
        if (apariciones[i-1] == aparicionesMax) sol.push_back(i);
    return sol;
}

bool resuelveCaso() {
    int numTiradas, valorMax;
    std::cin >> numTiradas >> valorMax;
    if (numTiradas == -1) return false;
    std::vector<int> v(numTiradas);
    for (int& i : v) std::cin >> i;
    std::vector<int> sol = resolver(v, valorMax);
    std::cout << sol[0];
    for (int i = 1; i < sol.size(); ++i )
        std::cout << ' ' << sol[i];
    std::cout << '\n';
    return true;
}

int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif

   // entrada con centinela
     while (resuelveCaso()) ;

    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
   #endif

    return 0;
}
```

9. Cálculo de la moda. Mejorando las carreteras

Mejorando las carreteras



El gobierno está intentando reducir el número de accidentes de carretera. Para ello ha elaborado un ambicioso plan de modernización de las vías en el que se pretende mejorar los tramos más peligrosos. Para detectar estos cuenta con los puntos kilométricos en que han ocurrido accidentes durante los últimos años. Ahora quiere obtener cuál es el punto kilométrico en que más accidentes han ocurrido para cada una de las carreteras del país.

Requisitos de implementación.

El problema pide calcular la moda de una colección de valores. En este caso, los valores no están acotados y por lo tanto no podemos utilizar un vector auxiliar para contar las apariciones de cada elemento. En lugar de esto, ordenaremos el vector y buscaremos el segmento máximo con todos los valores iguales.

La función resolver recibirá un vector con los datos leídos de la entrada. Ordenará los elementos y buscará el segmento máximo, devolviendo el valor que se repiten el número máximo de veces. Si existen dos valores que se repiten el número máximo de veces se elije el de más a la izquierda del vector.

Entrada

La entrada consta de una serie de casos de prueba. Cada caso consta de dos líneas, en la primera se muestra el número de accidentes ocurridos, y en la segunda se muestran los puntos kilométricos en que han ocurrido los accidentes. Estos se encuentran ordenados por la fecha en que ocurrió el accidente. El final de casos se marca con el valor -1.

Se garantiza que los valores de la secuencia son números enteros positivos que pueden almacenarse en una variable de tipo `int`. El número de elementos de la secuencia es siempre mayor que cero, pero no está acotado pudiendo ser "muy grande".

Salida

Para cada caso de prueba se escribe en una línea el punto kilométrico en que han ocurrido un mayor número de accidentes. En caso de que haya dos puntos kilométricos con el mismo número de accidentes se elegirá él que esté más cerca del comienzo de la carretera (el menor) ya que resulta más barato arreglar la carretera cuanto más cerca estemos del comienzo y tampoco es cuestión de gastar de más.

Entrada de ejemplo

```
7
5 3 8 5 10 10 5
6
900 50 900 200 50 200
1
12000
-1
```

Salida de ejemplo

```
5
50
12000
```

9.1. Objetivos del problema

- Conocer las diferentes formas de calcular la moda de una colección de elementos. Se conoce como moda al elemento que más veces se repite en la colección.
- Comprobar la importancia de que los elementos de una colección estén ordenados para obtener algoritmos eficientes.

9.2. Ideas generales.

- El problema pide calcular la moda de una colección de valores. Los valores no están acotados y por lo tanto no se puede utilizar un vector auxiliar para contar las apariciones de cada elemento.
- Ordenaremos los valores y después recorreremos el vector ordenado contando cuántas veces aparece cada valor y quedándonos con el máximo.
- Dependiendo de los datos concretos, el uso de un diccionario no ordenado podría dar una solución mejor al problema.

9.3. Algunas cuestiones sobre implementación.

- Se utiliza la función `sort` de la librería `algorithm` para ordenar los datos del vector de menor a mayor:

```
std::sort(v.begin(), v.end());
```

9.4. Ideas detalladas.

- Para calcular el segmento máximo de valores iguales se emplea el algoritmo de segmento máximo. Ver los problemas sobre este algoritmo.
- El vector se recorre con un bucle de izquierda a derecha.
- Se debe guardar información sobre el máximo número de valores consecutivos iguales hasta la posición que se ha recorrido en el vector, así como del valor. También se guarda información sobre el número de elementos iguales al valor de la posición que se está tratando (valor actual).
- Se recomienda realizar un bucle que en cada vuelta compruebe si el valor actual coincide con el anterior/siguiente. En caso de que coincida, el valor se considera *bueno*, se incrementa el contador de elementos iguales al que se está tratando y se comprueba si este contador ha superado al máximo que se tenía guardado. Si lo ha superado se debe actualizar el número máximo de veces que aparece un valor y el valor de que se trata.
- Si el valor actual no coincide con el valor anterior, el contador de elementos iguales al actual se debe inicializar a 1.

9.5. Coste de la solución

- En la solución del problema hay dos partes.
- Primero se ordena el vector. El algoritmo de ordenación tiene complejidad $\mathcal{O}(n \log n)$ siendo n el número de puntos kilométricos considerados en la entrada de datos.
- Después se ejecuta un bucle para buscar el máximo número de elementos iguales. El bucle da n vueltas, siendo n el número de puntos kilométricos. En cada vuelta se ejecuta un condicional, con asignaciones y comparaciones todas de coste constante. El coste del bucle es $\mathcal{O}(n)$ siendo n el número de puntos kilométricos.
- Por lo tanto el coste del algoritmo es el máximo entre el coste de ordenar y el coste del bucle: $\mathcal{O}(n \log n)$, siendo n el número de puntos kilométricos de la entrada.

9.6. Errores frecuentes.

- No emplear el esquema del algoritmo del segmento máximo propuesto. Si se actualiza el máximo cuando se termina un segmento de valores iguales, debe comprobarse si es necesario actualizarlo cuando se acaba el bucle, ya que puede ocurrir que el segmento máximo de elementos iguales se encuentre al final del vector y en ese caso no se habrá actualizado.

9.7. Implementación en C++.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>

int resolver(std::vector<int> & v){
    // Ordenar el vector
    std::sort(v.begin(), v.end());
    // Buscar el segmento maximo de elementos iguales
    int maxi = 1; int cont = 1; int km = v[0];
    for (int i = 1; i < v.size(); ++i){
        if (v[i] == v[i-1]) {
            ++cont;
            if (cont > maxi) { maxi = cont; km = v[i];}
        }
        else {
            cont = 1;
        }
    }
    return km;
}

bool resuelveCaso() {
    // lectura de los datos
    int numAccidentes;
    std::cin >> numAccidentes;
    if (numAccidentes == -1) return false;
    std::vector<int> v(numAccidentes);
    for (int& i : v) std::cin >> i;
    // Resolver el problema
    int sol = resolver(v);
    // Escribir la solucion
    std::cout << sol << '\n';
    return true;
}


int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif

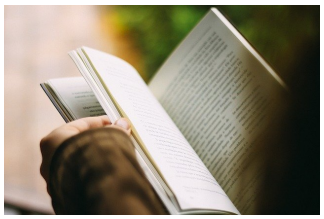
   // entrada concentinela
     while (resuelveCaso()) ;

    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
   #endif
```

```
    return 0;
}
```

10. Intervalos, releyendo un libro.

Releyendo un libro. Versión 1



Cuando tengo un rato libre me gusta coger un libro y releer alguna de las partes que más me gustaron cuando lo leí por primera vez. Normalmente, mientras voy leyendo doy una puntuación entre 0 y 10 a cada página, de forma que cuando quiero volver a leerlo se que páginas me gustaron más. Cero significa que no tengo interés en volver a leer esa página, mientras que un 10 indica que es una de las mejores partes de la obra. Una vez que empiezo a leer en una página, continuo con las páginas siguientes sin saltarme ninguna.

Para asegurarme de que leo la mejor parte he aprendido a estimar el número de páginas que me dará tiempo a leer en el rato que tengo. Siempre busco que las páginas sean lo más interesantes posible. El interés de varias páginas se calcula como la suma del interés de cada página. Si hay varios intervalos igual de interesantes elijo el que esté más avanzado en el libro. Recuerda que las páginas que leo son siempre consecutivas.

Requisitos de implementación.

La función que resuelve el problema debe tener un coste lineal respecto al número de páginas del libro. Puede implementarse más de un bucle, siempre que el coste sea lineal respecto al número de páginas del libro.

Entrada

La entrada consta de una serie de casos de prueba. Cada caso de prueba tiene dos líneas. En la primera se indica el número de páginas del libro ($1 < n < 200.000$), seguido de la cantidad de páginas que me dará tiempo a leer ($1 < L < n$). En la segunda línea se indica la puntuación que he dado a cada página del libro ($0 <= p <= 10$). La entrada finaliza con dos ceros.

Salida

Para cada caso de prueba se escribe la página en que debo empezar a leer el libro para que al leer L páginas empezando en ella, lea siempre las páginas más interesantes. Si existen dos bloques de páginas cuya suma de puntuaciones es igual, elegiré la que se encuentre más avanzada en el libro.

Entrada de ejemplo

```
6 3
0 5 3 4 1 2
7 2
6 4 5 6 0 0 8
5 4
7 7 7 7 7
6 2
5 6 7 6 7 5
0 0
```

Salida de ejemplo

```
1
2
1
3
```

Autor: Isabel Pita

1

10.1. Objetivos del problema

- Practicar problemas que buscan intervalos de longitud fija que cumplen una propiedad.
- Estudiar el esquema general utilizado en este tipo de problemas y las diferentes variantes que nos podemos encontrar.

10.2. Ideas generales.

- El problema se resuelve con un bucle inicial que recorre el primer intervalo. Este bucle permite inicializar las variable con los valores correspondientes al primer intervalo. Después se implementa un bucle principal donde se recorre el resto del vector. Al avanzar un dato se actualizan los valores de las variables eliminando el primer elemento del intervalo y añadiendo el nuevo elemento.

10.3. Algunas cuestiones sobre implementación.

- El programa debe tener dos bucles en secuencia. En el primero se recorren las primeras L páginas, siendo L la longitud del intervalo que tenemos tiempo de leer. En el segundo se recorren el resto de las páginas.
- En el segundo bucle, los índices deben recorrer todos los posibles intervalos excluyendo el primero. Los intervalos que debe recorrer este bucle son: [1..L+1), [2..L+2), [3..L+3) etc.
- La variable de control del segundo bucle puede ser el extremos izquierdo o el extremo derecho del intervalo. Si elegimos el extremo derecho tendremos el bucle:

```
for (int i = L; i < v.size(); ++i) {
    sumaAct -= v[i-L];
    sumaAct += v[i];
    if (sumaAct >= sumaMax){ // El intervalo es mejor
        posIni = i-L+1;
        sumaMax = sumaAct;
    }
}
```

  Mientras que si elegimos el extremo izquierdo tendremos:

```
for (int i = 0; i < v.size()-L; ++i) {
    sumaAct -= v[i];
    sumaAct += v[i+L];
    if (sumaAct >= sumaMax){ // El intervalo es mejor
        posIni = i+1;
        sumaMax = sumaAct;
    }
}
```

10.4. Ideas detalladas.

- La información necesaria para resolver el problema es la suma del intervalo que estamos considerando y la suma del intervalo máximo que hayamos encontrado, así como su posición inicial.
- En el primer bucle se calcula la suma del primer intervalo.
- A continuación se inicializa la suma máxima con la suma del primer intervalo y la posición inicial a cero.
- En el bucle que recorre el resto del vector restamos el interés de la primera página del intervalo y sumamos el interés de la página siguiente del intervalo.
- Nos piden el intervalo cuya suma sea máxima que se encuentra más a la derecha en el vector. Esto se consigue escogiendo el máximo más a la derecha cuando se pregunta en el bucle principal si la suma actual supera a la máxima ya guardada. Se observa que se modifica el inicio del intervalo si

la suma actual es mayor o igual. Es decir en caso de ser igual se modifica el comienzo del intervalo para que quede guardado el intervalo de más a la derecha.

```
if (sumaAct >= sumaMax){ // El intervalo es mejor
    posIni = i-L+1;
    sumaMax = sumaAct;
}
```

- Si se quisiera el intervalo de más a la izquierda utilizaríamos un menor estricto, de forma que si se encuentra una suma igual no se modifique el comienzo del intervalo.

10.5. Coste de la solución

El algoritmo tiene dos bucle. El primero se ejecuta tantas veces como el tamaño del intervalo. Las instrucciones del bucle incluyen una comparación, una suma y un incremento, todas ellas de coste constante, por lo tanto el coste de este bucle es el número de vueltas que da el bucle por el coste constante de cada vuelta y está en el orden $\mathcal{O}(p)$, siendo p el tamaño del intervalo.

El segundo bucle recorre todos los elementos del vector desde el final del primer intervalo hasta el final del vector. Por lo tanto el número de vueltas que da es `v.size() - p`. En cada vuelta se realizan 2 comparaciones, 6 operaciones aritméticas y un incremento, todas ellas operaciones de coste constante. Por lo tanto el coste de este bucle está en el orden de $\mathcal{O}(v.size() - p)$, que es equivalente a $\mathcal{O}(v.size())$.

10.6. Errores frecuentes.

1. No actualizar la suma máxima al acabar el primer bucle.
2. Errores derivados de utilizar un único bucle en el que se mezcla el tratamiento del primer intervalo con el tratamiento general del vector.

10.7. Modificaciones al problema.

- *Obtener el intervalo de tamaño fijo que incluya algunos valores.*
  - Por ejemplo nos piden el comienzo del intervalo de longitud L más a la izquierda del vector que incluya al menos dos ceros.
  - El esquema general es el visto en este problema.
  - En lugar de llevar calculada la suma del intervalo debemos llevar el número de ceros que tiene el intervalo. Al ir avanzando el intervalo en el segundo bucle comprobaremos si el primer valor del intervalo es un cero y si el nuevo elemento del intervalo es un cero para actualizar el contador del número de ceros.
- *Obtener el intervalo de tamaño fijo que incluya el máximo de los valores del vector.*
  - Por ejemplo nos piden el comienzo del intervalo de longitud L más a la izquierda del vector que incluya al menos un máximo.
  - El esquema general es el visto en el problema.
  - En primer lugar calcularemos cuál es el valor máximo del vector. El valor máximo puede estar repetido en el vector.
  - LLevaremos información sobre el valor máximo que hemos encontrado y la posición más a la derecha del intervalo en que hay un máximo.
  - En el primer bucle recorreremos el primer intervalo e iremos actualizando la posición del máximo si alguno de los valores del intervalo coincide con él.
  - Una vez finalizado el primer bucle actualizamos los valores del intervalo máximo encontrado. Hay que tener en cuenta que si el intervalo no contiene el valor máximo entonces no se debe actualizar el intervalo máximo.
  - En el bucle general recorremos el resto del vector actualizando la primera y última componente del intervalo. Se debe actualizar también la información sobre la posición del valor máximo más a la derecha. Si el intervalo no tiene un máximo entonces no se actualizará la información del intervalo máximo.

10.8. Implementación en C++.

```
#include <iostream>
#include <fstream>
#include <vector>

int resolver (std::vector<int> const& v, int L){
    // Primer intervalo
    int sumaMax = 0; int sumaAct = 0; int posIni;
    for (int i = 0; i < L; ++i) {
        sumaAct += v[i];
    }
    sumaMax = sumaAct; posIni = 0;}
    // Resto del vector
    for (int i = L; i < v.size(); ++i) {
        sumaAct -= v[i-L];
        sumaAct += v[i];
        if (sumaAct >= sumaMax){ // El intervalo es mejor
            posIni = i-L+1;
            sumaMax = sumaAct;
        }
    }
    return posIni;
}


// Resuelve un caso de prueba.
bool resuelveCaso() {
    // Lectura de los datos
    int numElem, L;
    std::cin >> numElem >> L;
    if (numElem == 0 && L == 0) return false;
    std::vector<int> v(numElem);
    for (int & i : v) std::cin >> i;
    // Resolver el problema
    int pos = resolver(v,L);
    // Escribir la solucion
    std::cout << pos << '\n';

    return true;
}

int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif

   // Entrada con centinela
    while (resuelveCaso())
        ;

    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
   #endif

    return 0;
}
```

11. Partición. Viajes a Marte.

Viajes a Marte



Los vuelos a Marte están cada vez más solicitados. Para poder cubrir la demanda, la compañía ha encargado unos vehículos espaciales con más filas de asientos. El constructor ha propuesto disminuir la distancia entre las filas de la parte posterior. De esta forma no será necesario hacer modificaciones en el diseño del aparato. El problema está en que en estas filas sólo podrán sentarse personas *bajitas*. La compañía, sin embargo, no quiere aumentar el presupuesto, por lo que ha decidido pedir a los viajeros su altura cuando compran el billete y a la hora de embarcar llama primero a los más *bajos* para que ocupen las filas posteriores.

Tu misión es dividir a los viajeros entre *bajos* y *altos* en función de la altura que te digan. Si la altura de algún viajero coincide con la altura establecida se le considerará *bajo*. Debes también indicar el número de viajeros *bajos* del vuelo.

Entrada

La entrada consta de una serie de casos de prueba. Cada caso de prueba consta de $n+1$ líneas. En la primera se indica el número n de viajeros y la altura establecida para diferenciarlos. En cada una de las líneas siguientes aparece el nombre de un viajero y su altura.

El número de viajeros es mayor o igual que 0 y menor que 150.000. Los nombres de los viajeros son cadenas de caracteres sin blancos. La altura son números enteros positivos.

Salida

Para cada caso de prueba se escriben dos líneas. La primera comienza con `Bajos:` seguido de los nombres de los viajeros *bajos*. La segunda comienza con `Altos:` seguido de los nombres de los viajeros *altos*.

Para poder comparar tu salida con la del juez los nombres de los viajeros se mostrarán en orden alfabético, aunque esto no debe alterar el algoritmo empleado para resolver el problema, sino solo la salida de los datos.

Entrada de ejemplo

```
5 170
Rosa 150
Olga 175
Maria 180
Lucia 170
Ana 150
4 180
Daniel 190
Jose 200
Antonio 190
Roberto 200
2 150
Beatriz 140
Gonzalo 120
```

Salida de ejemplo

```
Bajos: Ana Lucia Rosa
Altos: Maria Olga
Bajos:
Altos: Antonio Daniel Jose Roberto
Bajos: Beatriz Gonzalo
Altos:
```

Autor: Isabel Pita

11.1. Objetivos del problema

- Practicar el algoritmo de partición cuando los datos del vector se dividen en dos partes.
- Este algoritmo es muy importante ya que se utiliza en el algoritmo de *quick sort* para ordenar secuencias de elementos.

11.2. Ideas generales.

- En el problema nos piden modificar el vector, de forma que en la parte izquierda queden todos los elementos menores o iguales que un valor dado y en la parte derecha todos los mayores. Debemos calcular también la posición que divide al vector.
- LLevaremos dos índices. El primer índice marca las posiciones del vector tales que a la izquierda del índice todos los valores son menores o iguales que el valor dado. El segundo índice marca las posiciones del vector tales que todos los valores a su derecha son mayores que el valor dado. Los valores entre los dos índices son los que todavía no se han tratado y por lo tanto pueden ser mayores o menores que el valor dado.

11.3. Algunas cuestiones sobre implementación.

- El vector se pasa por referencia, ya que se debe modificar en la función.
- En los datos de entrada, los nombres de los viajeros son cadenas de caracteres sin blancos por lo que se pueden leer utilizando `std::cin` en una variable de tipo `std::string`.
- En el vector debemos guardar el nombre y la altura de cada viajero. Esto se puede hacer con:
  - Un vector de pares `std::vector<std::pair<std::string, int>>`. En este caso, al ordenar el vector con la función `sort`, se utiliza el operador menor definido para el tipo `pair`. Este operador ordena los valores según el orden de la primera componente y si el valor de la primera componente es igual por el valor de la segunda.
  - Un `struct` con dos campos que representen los dos valores:

    ```
    struct tInfo {
      std::string nombre;
      int altura;
    };
    ```

    En este caso se debe sobrecargar el operador menor para el tipo `tInfo` definido en el `struct` para que la función `sort` lo pueda utilizar.

    ```
    bool operator< (tInfo d1, tInfo d2) {
      return (d1.nombre < d2.nombre) || (d1.nombre == d2.nombre && d1.altura < d2.altura);
    }
    ```

  - Para que el juez pueda corregir el problema se pide que la salida se muestre ordenada por orden alfabético. Sin embargo esto no debe afectar al algoritmo implementado. Por ello, una vez que la función devuelve el vector con los elementos menores a la izquierda de la posición `p` devuelta por la función y los elementos mayores desde la posición `p` hasta el final del vector se ordenan ambas mitades con la función `sort` de la librería `algorithm`.

    ```
    std::sort(v.begin(), v.begin()+p);
    std::sort(v.begin()+p, v.end());
    ```

11.4. Ideas detalladas.

- Se inicializa un índice a la primera posición del vector y otro a la última posición del vector.
- Se implementará un solo bucle que se ejecuta mientras no se crucen los dos índices.
- En cada vuelta del bucle se comprueba si el elemento correspondiente al primer índice está bien colocado, en caso afirmativo se incrementa este índice, si no, si el elemento correspondiente al segundo índice está bien colocado se decrementa este índice, y si no es que ambos elementos están más colocados. En este caso se intercambian los valores y se mueven los dos índices.

11.5. Coste de la solución

- En la implementación que se muestra al final del ejercicio y que sigue las ideas presentadas anteriormente, el bucle principal está controlado por dos índices. En cada vuelta del bucle se modifica uno de los índices o los dos. Por lo tanto el número máximo de vueltas que da el bucle coincide con el número de elementos del vector.
- En cada vuelta del bucle se realizan dos comparaciones, un intercambio de variables y cuatro incrementos o decrementos. Todas estas operaciones tienen coste constante (el intercambio es de dos pares con tipos `std::string` e `int`, el tipo `std::string` se intercambia con coste constante debido a que soporta la semántica de movimiento y el tipo `int` se intercambia tambien con coste constante por ser un tipo básico). Por lo tanto el coste del algoritmo esta en el orden $\mathcal{O}(n)$ siendo n el número de viajeros del vuelo.

11.6. Errores frecuentes.

No se conocen

11.7. Implementación en C++.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <algorithm>

using psi = std::pair<std::string, int>;  // Nombre del viajero y su altura

int particion (std::vector<psi> & v, int altura) {
    int p = 0, q = (int)v.size()-1;
    while (p <= q) {
        if (v[p].second <= altura)  ++p; // elemento del indice izquierdo correcto
        else if (v[q].second > altura) --q;  // elemento del indice derecho correcto
        else {  // Ambos elementos fuera de sitio
            std::swap(v[p],v[q]);
            ++p; --q;
        }
    }
    return p;
}


// Resuelve un caso de prueba, leyendo de la entrada la
// configuracion, y escribiendo la respuesta
bool resuelveCaso() {
    int numViajeros, altura;
    std::cin >> numViajeros;
    if (!std::cin) return false;
    std::cin >> altura;
    std::vector<psi> v(numViajeros); // nombre y altura
    for (psi & n : v) {
        std::cin  >> n.first >> n.second;
    }
    int p = particion(v, altura);
    // Ordena cada parte para la salida de datos
    std::sort(v.begin(), v.begin()+p);
    std::sort(v.begin()+p, v.end());
    // Escribe la primera mitad
    if (p > 0) std::cout << v[0].first;
    for (int i = 1; i < p; ++i){
```

```
        std::cout << ' ' << v[i].first ;
    }
    std::cout << '\n';
    // Escribe la segunda parte
    if (p < v.size()) std::cout << v[p].first;
    for (int i = p+1; i < v.size(); ++i){
        std::cout << ' ' << v[i].first ;
    }
    std::cout << '\n';
    std::cout << "---\n";
    return true;
}

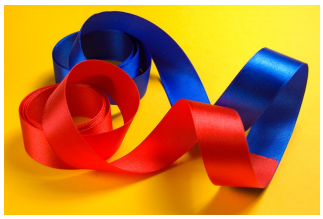

int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif
   // Entrada con casos ilimitados
    while (resuelveCaso()) {} //Resolvemos todos los casos

    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
   #endif

    return 0;
}
```

12. Partición. Cintas de colores.

Cintas de colores



Tengo una caja llena de cintas de colores rojas, azules y verdes. Las he estado midiendo y ahora quiero ordenarlas por colores, para poder saber fácilmente las longitudes que tengo de cada color.

Dada una lista de todas las cintas con sus longitudes, sin ningún orden, debes ordenarlas de forma que al principio de la lista aparezcan las cintas azules, a continuación las verdes y por último las rojas. Dentro de cada grupo no es necesario que las cintas aparezcan en un orden determinado.

Requisitos de implementación.

Se debe implementar una función que reciba un vector con las cintas identificadas por su color y su longitud y modifique el vector para dejar al principio las cintas azules, en el medio las cintas verdes y por último las cintas rojas. La función devolverá dos índices indicando donde empiezan y terminan las cintas verdes. El coste de la función debe ser lineal en el número de cintas.

Debe emplearse el algoritmo de partición con dos índices.

Entrada

La entrada consta de una serie de casos de prueba. Cada caso de prueba consta de dos líneas. En la primera se indica el número n de cintas. En la siguiente se muestra el color de cada cinta seguido de su longitud.

El número de cintas es mayor o igual que 0 y menor que 300.000. Los colores de las cintas se identifican por su primer carácter: `a` para las cintas azules, `v` para las cintas verdes y `r` para las cintas rojas. Las longitudes son números enteros positivos.

Salida

Para cada caso de prueba se escriben tres líneas. La primera comienza con `Azules:` seguido de las longitudes de las cintas azules. La segunda comienza con `Verdes:` seguido de las longitudes de las cintas verdes. La tercera comienza con `Rojas:` seguido de las longitudes de las cintas rojas.

Para poder comparar tu salida con la del juez las longitudes se mostrarán en orden de menor a mayor, aunque esto no debe alterar el algoritmo empleado para resolver el problema, sino solo la salida de los datos.

Entrada de ejemplo

```
6
v 5 r 1 a 3 a 7 v 3 r 6
3
r 4 r 7 r 1
5
a 6 a 2 v 1 v 3 a 5
4
r 5 r 7 a 9 a 3
```

Salida de ejemplo

```
Azules: 3 7
Verdes: 3 5
Rojas: 1 6
Azules:
Verdes:
```

```
Rojas: 1 4 7
Azules: 2 5 6
Verdes: 1 3
Rojas:
Azules: 3 9
Verdes:
Rojas: 5 7
```

12.1. Objetivos del problema

- Practicar el algoritmo de partición cuando los datos del vector se dividen en tres partes.
- Este algoritmo es muy importante ya que se utiliza en el algoritmo de *quick sort* para ordenar secuencias de elementos.

12.2. Ideas generales.

- En el problema nos piden modificar el vector, de forma que en la parte izquierda queden todas las cintas de color azul, en la parte del centro las cintas de color verde y por último las cintas de color rojo. Debemos calcular también las posiciones que separan las diferentes partes.
- Al dividir el vector en tres partes es necesario mantener tres índices. El primer índice marca las posiciones del vector tales que a la izquierda del índice todos los valores son cintas de color azul. El segundo índice marca las posiciones del vector tales que todas las cintas desde el primer índice hasta el segundo son de color verde. El tercer índice marca las posiciones del vector tales que todas las cintas a su derecha son de color rojo. Los valores entre el segundo y tercer índice son los que todavía no se han tratado y por lo tanto pueden ser cintas de cualquier color.

12.3. Algunas cuestiones sobre implementación.

- El vector se pasa por referencia, ya que se debe modificar en la función.
- En el vector debemos guardar el color y la longitud de cada cinta. Esto se puede hacer con:
  - Un vector de pares `std::vector<std::pair<char, int>>`. En este caso, al ordenar el vector con la función `sort`, se utiliza el operador menor definido para el tipo `pair`. Este operador ordena los valores según el orden de la primera componente y si el valor de la primera componente es igual por el valor de la segunda.
  - Un `struct` con dos campos que representen los dos valores:

    ```
    struct tInfo {
      char color;
      int longitud;
    };
    ```

    En este caso se debe sobrecargar el operador menor para el tipo `tInfo` definido en el `struct` para que la función `sort` lo pueda utilizar.

    ```
    bool operator< (tInfo d1, tInfo d2) {
      return (d1.longitud < d2.longitud);
    }
    ```

- Para que el juez pueda corregir el problema se pide que la salida se muestre ordenada por la longitud de las cintas. Sin embargo esto no debe afectar al algoritmo implementado. Por ello, una vez que la función devuelve el vector con las cintas azules a la izquierda de la posición `p` (valor de retorno de la función), las cintas verdes entre la posición `p` y la posición `q` (valor de retorno de la función) y las cintas de color rojo desde la posición `q` hasta el final del vector se ordena cada parte con la función `sort` de la librería `algorithm`.

    ```
    std::sort(v.begin(), v.begin()+p);
    std::sort(v.begin()+p, v.begin()+1+q);
    std::sort(v.begin()+1+q, v.end());
    ```

- El índice `q` indica la última componente de color verde. Por ello si todas las cintas son rojas este índice toma el valor -1. Al ordenar la parte verde y la parte roja debemos escribir el final y el principio de la parte a ordenar respectivamente como

    ```
    std::sort(v.begin()+p, v.begin()+1+q);
    std::sort(v.begin()+1+q, v.end());
    ```

  El motivo es que la expresión `v.begin() + q` si el valor de `q` es -1 es erróneo.

12.4. Ideas detalladas.

- Se inicializan dos índices `i` y `k` a la posición cero y un índice `q` a la última posición del vector. Los valores a la izquierda de `i` serán de color azul, los valores entre `i` y `k` serán los de color verde y los valores a la derecha de `q` serán los de color rojo. Los valores entre `k` y `q` son los valores que todavía no se han tratado.
- Se implementará un solo bucle que se ejecuta mientras no se crucen los dos índices `k` y `q`.
- En cada vuelta del bucle se comprueba si el elemento correspondiente al índice `k` es de color verde, en caso afirmativo está bien colocado y se incrementa el índice `k`. Si es de color azul se intercambia con el elemento de la posición `p` y se incrementan los índices `p` y `q`. Si es de color rojo se intercambia con el elemento de la posición `q` y se decrementa el índice `q`. Observar que en este último caso no se incrementa el valor de `k` porque no se ha comprobado el color del elemento que estaba en la posición `q` y por lo tanto no se puede considerar que ya está tratado.

12.5. Coste de la solución

- En la implementación que se muestra al final del ejercicio y que sigue las ideas presentadas anteriormente, el bucle principal está controlado por dos índices `k` y `q`. En cada vuelta del bucle se modifica uno de los índices. Por lo tanto el número máximo de vueltas que da el bucle coincide con el número de elementos del vector.
- En cada vuelta del bucle se realizan comparaciones, intercambio de variables e incrementos o decrementos. Todas estas operaciones tienen coste constante (el intercambio es de dos pares con tipos `std::char` e `int`), por lo tanto el coste del algoritmo está en el orden $\mathcal{O}(n)$ siendo n el número de cintas.

12.6. Errores frecuentes.

- El índice `q` indica la última componente de color verde. Por ello si todas las cintas son rojas este índice toma el valor -1. Al ordenar la parte verde y la parte roja debemos escribir el final y el principio de la parte a ordenar respectivamente como

  ```
  std::sort(v.begin()+p, v.begin()+1+q);
  std::sort(v.begin()+1+q, v.end());
  ```

  El motivo es que la expresión `v.begin() + q` si el valor de `q` es -1 es erróneo, por lo tanto le sumamos primero uno al valor de `v.begin()` y de esta forma luego se puede restar. .

12.7. Implementación en C++.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>

using psi = std::pair<char, int>;

void particion (std::vector<psi> & v, int &p, int &q) {
    p = 0; q = (int)v.size()-1; int k = 0;
    while (k <= q ) {
        if (v[k].first == 'v') ++k;
        else if ( v[k].first == 'a')  {
            std::swap(v[p],v[k]);
            ++p; ++k;
        }
        else {
            std::swap(v[q],v[k]);
            --q;
        }
```

```
    }
}


bool resuelveCaso() {
    // lectura de los datos de entrada
    int numCintas;
    std::cin >> numCintas;
    if (!std::cin) return false;
    std::vector<psi> v(numCintas); // nombre y altura
    for (psi & n : v) {
        std::cin  >> n.first >> n.second;
    }
    // resuelve el problema
    int p, q;
    particion(v, p, q);
    // Ordena cada parte para la salida de datos
    std::sort(v.begin(), v.begin()+p);
    std::sort(v.begin()+p, v.begin()+1+q); // Debe ser 1+q para evitar v.begin()-1 cuando q vale
    std::sort(v.begin()+1+q, v.end());
    // Escribe las cintas azules
    std::cout << "Azules:";
    for (int i = 0; i < p; ++i){
        std::cout << ' ' << v[i].second ;
    }
    std::cout << '\n';
    // Escribe la segunda parte
    std::cout << "Verdes:";
    for (int i = p; i <= q; ++i){
        std::cout << ' ' << v[i].second ;
    }
    std::cout << '\n';
    // Escribe la tercera parte
    std::cout << "Rojas:";
    for (int i = q+1; i < v.size(); ++i){
        std::cout << ' ' << v[i].second ;
    }
    std::cout << '\n';
    return true;
}

int main() {
    // Para la entrada por fichero.
   #ifndef DOMJUDGE
    std::ifstream in("datos.txt");
    auto cinbuf = std::cin.rdbuf(in.rdbuf());
   #endif
   // Entrada con casos ilimitados
    while (resuelveCaso()) ; //Resolvemos todos los casos

    // Para restablecer entrada.
   #ifndef DOMJUDGE
      std::cin.rdbuf(cinbuf);
      system("PAUSE");
   #endif

    return 0;
}
```